

Abstract

The rise of large language models (LLMs) has significantly influenced various fields, including natural language processing (NLP) and image generation, by making complex computational tasks more accessible. Despite their remarkable generative capabilities, a fundamental question remains regarding their level of *understanding*, particularly in structured domains such as SQL, where precise logic and syntactic accuracy are essential. This work evaluates the extent to which LLMs comprehend SQL by assessing their performance on key tasks, including syntax error detection, missing token identification, query performance prediction, query equivalence checking, and query explanation. These tasks collectively examine the models' ability to recognize patterns, maintain context awareness, interpret semantics, and ensure logical coherence—capabilities that are critical for genuine SQL understanding.

To enable a rigorous evaluation, we construct labeled datasets from well-established SQL workloads and conduct extensive experiments on state-of-the-art LLMs. Our analysis specifically investigates how query complexity and distinct syntactic features impact model performance. The results indicate that while models such as GPT4 excel in tasks that rely on pattern recognition and contextual awareness, they exhibit persistent difficulties in deeper semantic understanding and logical consistency. These challenges are particularly evident in tasks such as accurately predicting query performance and verifying query equivalence.

This gap suggests that current LLMs, despite their syntactic and structural proficiency, lack the ability to integrate deeper semantic reasoning required for comprehensive SQL comprehension. Our findings underscore the need for future advancements in LLMs to focus on improving their reasoning abilities and their capacity to incorporate domain-specific knowledge. Enhancing these aspects would enable a transition from syntactic fluency to a more logic-driven understanding, thereby unlocking the full potential of SQL in various computational applications.

Keywords: LLMs, NLP, SQL tasks, SQL understanding.

Summary for Lay Audience

In recent years, artificial intelligence has made significant progress, especially through the development of large language models (LLMs) like ChatGPT. These models can generate human-like text and have been used in areas such as writing, translation, and even creating images. However, while they are good at mimicking language, it's not clear whether they truly "understand" what they are doing—especially in areas that require strict logic, like programming languages.

This research focuses on one such language: SQL, which is used to interact with databases. Writing correct SQL queries requires not just good grammar, but also deep logical thinking. To test how well LLMs understand SQL, this study examines their performance on five key tasks: identifying errors, spotting missing pieces, predicting how long a query will take to run, checking whether two queries are equivalent, and explaining the logic behind queries.

To evaluate these abilities, the study uses real SQL data and designs experiments using leading LLMs. The results show that while models like GPT-4 can detect patterns and understand context well, they struggle with tasks that require deeper reasoning—such as accurately predicting performance or judging whether two queries do the same thing.

This finding suggests that even the most advanced LLMs are still limited in their understanding of structured, logic-based languages like SQL. They may appear intelligent but often lack true comprehension when deeper logic is required. The research concludes that if we want LLMs to be more reliable in professional or technical settings, especially those involving databases, future models must be trained not only on language patterns but also on reasoning skills and domain-specific knowledge.

Dedication

To my son.

Your arrival gave me strength I never knew I had.
I carried you through my master's, so it's your journey too —
and this work is dedicated to you.

Acknowledgements

First and foremost, I would like to thank my supervisor, Dr. Mostafa Milani, for his continuous support, insightful guidance, and encouragement throughout my research journey. His mentorship has been invaluable.

I am deeply grateful to my mother, whose unwavering belief in me has always been my source of strength. To my husband, Shekh Tariful Huda, thank you for your constant support, patience, and love during every step of this academic pursuit.

Finally, this thesis is based in part on joint work with Dr. Rachel Pottinger and Dr. Fei Chiang, as published in the 28th International Conference on Extending Database Technology (EDBT), 25–28 March, Barcelona, 2025.

Contents

Abstract	ii
Summary for Lay Audience	iii
Dedication	iv
Acknowledgements	v
Contents	vi
List of Tables	viii
List of Figures	ix
1 Introduction	1
1.1 SQL Tasks	3
1.1.1 Syntax Error Identification	4
1.1.2 Missing Token Identification.	5
1.1.3 Query Performance Estimation	5
1.1.4 Query Equivalence	6
1.1.5 Query Explainability	7
1.2 Thesis Contributions and Structure	8
2 Related Work	10
2.1 LLMs for Data Processing and Entity Matching	10
2.2 LLMs for SQL and Structured Data Processing	11
2.3 Prompting Techniques for Enhancing LLM Reasoning	11
2.4 Evaluating and Comparing LLM Capabilities	11
3 Query-Workloads	13
3.1 Syntactic Properties of SQL Queries	15
4 Experimental Framework	21
4.1 SQL Tasks	21
4.1.1 Binary Tasks	21
4.1.2 Multi-Class Tasks	26
4.1.3 Query Explanation	27

4.2	Data Preparation and Label Generation	27
4.3	Large Language Models	29
4.4	Refining LLM Interactions	30
4.5	Baselines for Experimental Evaluation	32
5	Experimental Results	34
5.1	Syntax Error Tasks	34
5.2	Missing Token Tasks	36
5.3	Query Performance Prediction	40
5.4	Query Equivalence	42
5.5	Case Study: Query Explanation	44
5.6	Reflections on SQL Understanding	46
5.7	Reflections and Broader Insights	46
5.7.1	Actionable Insights and Future Directions	47
6	Conclusion	48
6.1	Future Work	49
	Bibliography	51
	Curriculum Vitae	57

List of Tables

1.1	Skill-to-SQL task mapping	7
3.1	Workload statistics overview	14
5.1	Accuracy in <code>syntax_error</code> and <code>syntax_error_type</code>	34
5.2	Accuracy for <code>miss_token</code> and <code>miss_token_type</code>	38
5.3	MAE and Hit Rate (HR) for <code>miss_token_loc</code>	41
5.4	Acc. for <code>performance_pred</code>	41
5.5	Accuracy in <code>query_equiv</code> and <code>query_equiv_type</code>	43

List of Figures

3.1	SDSS Statistics	15
3.2	SQLShare Statistics	16
3.3	Join Order Statistics	17
3.4	Statistics for Spider workload	18
3.5	column_count for SDSS, SQLShare and Join-Order	18
3.6	join_count for all workloads	19
3.7	Pairwise correlations between query properties for each workload	20
4.1	Elapsed time of sampled SDSS queries	26
5.1	Relationship between word_count and model failure in syntax_error for SDSS. The three numbers (e.g., 65.45, 57.00, 140) represent the average and median query length and the number of queries in the category (TP). The orange scatter points represent queries plotted on the y-axis by their length, showing the length distribution per category.	35
5.2	Relationship between syntax error type and FN in syntax_error.	37
5.3	LLMs' failure in miss_token for SQLShare.	39
5.4	Relationship between missing token type and FN in miss_token.	40
5.5	MistralAI's failure in performance_pred	42
5.6	Failure in performance_pred	42
5.7	word_count and LLM failures in query_equiv.	44
5.8	predicate_count and LLM failure in query_equiv.	44

Chapter 1

Introduction

The rapid advancement of LLMs has significantly transformed various fields, improving the accessibility and efficiency of computational and data-driven tasks. In domains such as natural language processing (NLP) and image generation, these models can produce human-like text and generate highly realistic images, demonstrating remarkable generative capabilities. Despite their success, LLMs do not possess human-level *understanding* in the conventional sense. However, their ability to solve tasks beyond their direct training suggests a certain degree of *comprehension* [58, 27, 47]. In this context, *understanding* refers to an LLM’s capacity to perform fundamental tasks with accuracy and consistency, often matching or even surpassing human performance in specific scenarios.

This level of proficiency can be evaluated based on a set of fundamental *skills* that contribute to *understanding* and practical application. Regular assessments help identify weaknesses, track improvements, and ensure continuous progress. These skills include *recognition*, *semantics*, *context*, and *coherence*, each playing a crucial role in a model’s ability to perform complex tasks accurately:

- *Recognition* refers to the ability to identify relevant entities or objects of interest. For example, in image generation, recognition involves distinguishing between different objects and scenes, requiring sophisticated algorithms that analyze visual features and contextual cues. This skill is essential in critical applications such as autonomous driving and medical imaging, where precise identification can have life-altering consequences.
- *Semantics* concerns the construction and interpretation of meaning. For instance, the meaning of a red octagon universally signals a stop, while in NLP, semantic understanding involves grasping the meaning of words and phrases in context. Effective semantic comprehension also extends to recognizing cultural nuances that influence interpretation, which is particularly important in multilingual and cross-cultural communication.
- *Context* defines the conditions under which meaning is interpreted. In NLP, this includes resolving ambiguities when words or phrases have multiple meanings depending on the surrounding text, understanding intent (e.g., detecting sarcasm, politeness, or indirect requests), and handling out-of-distribution scenarios by identifying elements that deviate from expected patterns. Strong contextual awareness is necessary for tasks such as dialogue systems, machine translation, and reasoning over structured data.

- *Coherence* ensures logical consistency and meaningful interconnections between different elements. In image generation, coherence ensures that objects are positioned realistically relative to each other. In text-based tasks, it refers to the logical flow of ideas across words, sentences, and paragraphs, ensuring that information is presented in a structured and meaningful manner.

Achieving true *understanding* requires LLMs to demonstrate increasing proficiency in these skills and to perform task-specific operations with accuracy and reliability. A deeper examination of LLMs' *understanding* is critical for ensuring dependable performance in real-world applications, where precision and consistency are essential.

For example, in image generation, *understanding* involves several key abilities, including recognizing and distinguishing between different objects, maintaining object coherence (ensuring that objects are placed in realistic positions relative to each other), interpreting scene context (identifying relationships between objects and their surroundings), and handling out-of-distribution elements (detecting when an object or scenario does not conform to familiar patterns). These capabilities allow models to generate images that are not only visually appealing but also logically consistent with real-world expectations.

Similarly, in NLP, *understanding* encompasses multiple layers of language comprehension. This includes grasping *semantics* (interpreting the meaning of words and phrases in context), maintaining *contextual awareness* (tracking references and dependencies across longer text passages), resolving *ambiguities* (correctly interpreting words or phrases with multiple possible meanings based on their context), making *logical inferences* (drawing conclusions based on textual evidence), and comprehending *intent* (recognizing the underlying purpose behind a speaker's words, such as detecting sarcasm, indirect speech, or politeness). Unlike simple text generation, these tasks require deeper reasoning and an ability to process language in a more human-like manner.

Achieving *understanding* is not merely about producing coherent outputs but about performing these deeper, task-specific operations with accuracy and consistency. Assessing the extent to which LLMs possess these abilities is crucial for ensuring their reliability in real-world applications, where precision and logical consistency are essential.

However, despite significant advancements, growing evidence suggests that these models often lack what would be considered a true understanding of the content they generate [58, 27, 47]. While they can produce text and images that appear meaningful, their underlying reasoning remains limited, highlighting the need for further research into enhancing their comprehension and decision-making capabilities.

A stronger *understanding* reduces the risk of errors, increasing trust and safety, particularly in high-stakes environments. This improvement is critical in domains where precise comprehension is essential, enabling AI systems to operate more reliably and effectively across a broader range of applications.

In the context of image generation versus understanding, while LLMs can generate visually appealing images, they often struggle with maintaining object-level accuracy and preserving correct spatial relationships between objects [36]. For example, deep generative models frequently misinterpret common objects, confusing similar categories such as dogs and horses, or failing to distinguish between unrelated elements, such as house numbers and abstract patterns [39]. These limitations underscore the gap between surface-level content generation and

proficiency in the core tasks that define *understanding* in this domain, including object recognition, spatial coherence, and scene interpretation.

A similar challenge exists in NLP. Despite their strong generative capabilities, state-of-the-art models often rely on superficial correlations rather than truly grasping the underlying semantics of language [42]. For instance, LLMs may perform well on standardized benchmarks by exploiting statistical patterns, yet they struggle with real-world linguistic tasks requiring deeper comprehension, such as resolving ambiguous references, maintaining contextual awareness, and generating logically consistent responses in complex dialogues [24]. In software development, while LLMs can assist in code generation, they frequently fail to understand intricate or unfamiliar codebases, particularly when dealing with abstract concepts or non-standard programming patterns [40]. These challenges highlight a fundamental issue: the ability to generate content—whether text, images, or code—is insufficient without proficiency in underlying tasks such as semantic interpretation, logical reasoning, and intent comprehension, all of which indicate a more meaningful engagement with the material.

This gap between content generation and true understanding is particularly evident in *data management*, where LLMs’ limitations in handling structured data have been well-documented [55]. Benchmarks designed to assess LLMs on tasks such as cell lookup, row retrieval, and dataset size estimation reveal a recurring trend: while models perform well on simple tasks with explicit instructions, they struggle when faced with complex tasks requiring a deeper structural comprehension. These deficiencies reinforce the ongoing challenge of moving beyond content generation toward true domain-specific expertise, which is necessary for achieving a higher level of *understanding* in any field.

Toward this goal, we investigate how LLMs can be leveraged in data management applications, specifically their ability to perform SQL-related tasks. Our study extends beyond content generation; it systematically evaluates specific SQL tasks that reflect the fundamental skills required for *understanding*. We explore a key question regarding the role of LLMs in SQL processing: *To what extent do LLMs “understand” SQL?* Similar to other domains, such as image generation and NLP, assessing a model’s understanding involves more than testing its ability to generate syntactically correct outputs. Instead, it requires evaluating its proficiency in performing essential SQL-related tasks to determine the depth of its comprehension. Addressing this question is critical because if LLMs exhibit a strong understanding of SQL, they could enhance a wide range of data management applications where SQL plays a fundamental role. To provide a thorough assessment, this work focuses on key SQL tasks, enabling a detailed evaluation of how well LLMs grasp SQL concepts and apply them in practical scenarios.

1.1 SQL Tasks

We propose a set of core SQL tasks designed to assess the depth of LLMs’ SQL *understanding*. These tasks are fundamental to a wide range of SQL-related applications and are structured to progress from simpler tasks, such as syntax error detection, to more complex ones, such as query equivalence checking and explanation. SQL users, from novices to experts, regularly engage in tasks that range from identifying syntax errors to estimating query performance, determining query equivalence, and providing meaningful explanations. To evaluate LLMs’ proficiency, we assess their ability to perform these tasks in an increasing order of difficulty,

ensuring that each level reflects a higher degree of SQL comprehension and skill.

1.1.1 Syntax Error Identification

Detecting syntax errors in SQL queries requires varying levels of *understanding*, depending on the complexity of the errors. Basic syntax errors, such as missing parentheses, unclosed brackets, or misplaced operators, are often straightforward and can be identified by checking the query against predefined syntax rules. These errors do not require a deep understanding of SQL structure or query semantics.

In contrast, advanced syntax errors demand a more refined comprehension of SQL execution and its logical structure. Identifying such errors requires analyzing the relationships between different query components and understanding their intended functionality. Examples include:

- *Misalignment of attributes:* Errors occur when attributes referenced in one clause (e.g., `SELECT` or `WHERE`) do not align with those required by other clauses such as `GROUP BY` or `ORDER BY`. For instance, selecting a column that is not aggregated in a query using `GROUP BY` can lead to execution errors or incorrect results. Detecting such misalignments requires understanding both SQL syntax and the logical dependencies between query components.
- *Aggregation function misapplication:* These errors involve improper use of aggregation functions, such as applying an aggregation function in a `SELECT` clause without a corresponding `GROUP BY` clause. Another common mistake is using the `HAVING` clause to filter non-aggregated columns, which can result in runtime errors. Identifying such mistakes requires a deeper understanding of how aggregation functions interact with grouping and filtering mechanisms in SQL.
- *Incompatible attribute types between queries:* These errors arise when there are mismatches in data types between attributes used in subqueries and the main query. For example, comparing a string attribute from a subquery to an integer attribute in the main query may cause execution failures or unexpected results. Detecting these errors requires analyzing data types and ensuring compatibility between different query components.
- *Invalid Join Operations:* Errors in join operations occur when tables are incorrectly linked or when inappropriate join types are used. This includes cases where a query joins tables on unrelated or mismatched attributes, leading to incorrect or excessively large result sets. Another common issue is selecting a join type that does not meet the query's logical requirements, such as using an `INNER JOIN` when an `OUTER JOIN` is needed to retain necessary data. Identifying these errors requires understanding table relationships and the impact of different join strategies.

Detecting these advanced errors goes beyond basic syntactical validation and requires an in-depth semantic analysis of the query's intent and the underlying database schema. This level of SQL *understanding* is crucial for ensuring query correctness and efficiency, particularly in complex database environments where errors can lead to incorrect results or performance issues.

1.1.2 Missing Token Identification.

Detecting missing tokens is a critical capability for applications such as query recommendation, where token imputation and query auto-completion play a key role in improving user experience [26, 67]. These systems rely on accurately identifying and filling in missing tokens to streamline query formulation, enhance correctness, and reduce user effort.

We evaluate not only the ability to recognize when a token is missing but also the precision in identifying its exact location and type. Missing tokens can vary in nature, including essential SQL components such as:

- *Keywords*: Fundamental SQL commands such as SELECT, WHERE, JOIN, and others, which define the operations performed by the query.
- *Table Names*: References to database tables required for query execution that may be omitted accidentally.
- *Column Names*: Specific attributes within tables that must be explicitly mentioned for operations such as selection, sorting, and filtering.
- *Aliases*: Shortened names assigned to tables or columns, commonly used in joins and complex queries to improve readability, but which can be overlooked.
- *Literal Values*: Specific values used in conditions or assignments within the query that might be unintentionally omitted.

In query recommendation systems, the ability to detect and resolve missing tokens is essential. By analyzing gaps in query syntax, these systems provide real-time auto-completion suggestions, allowing users to construct complete and syntactically valid SQL queries with greater efficiency. Suggesting missing components such as keywords, table names, or column names helps reduce errors and enhances the user's ability to formulate queries accurately.

Furthermore, pinpointing the exact location and type of a missing token enables more context-aware recommendations. This precision improves the effectiveness of query recommendation systems, making them more robust and adaptable to different SQL query structures. By offering relevant and timely suggestions, these systems contribute to a smoother query-building experience, ensuring that users can efficiently construct correct and well-structured database queries.

1.1.3 Query Performance Estimation

Estimating the runtime performance of an SQL query based only on its textual representation is a challenging task. Query execution time depends on multiple factors, including the database schema, the specific data instances, and the overall system workload [67]. These factors influence how efficiently a database management system (DBMS) processes a query, making performance estimation a non-trivial problem.

Recent studies using publicly available query workloads have shown that query complexity plays a significant role in execution cost [62, 18, 29]. Longer queries that involve multiple joins

and numerous filtering conditions generally require more computation, leading to higher execution times. Queries that process large tables or involve expensive operations, such as sorting or aggregation, further increase execution costs. Understanding these patterns is essential for optimizing query performance and database efficiency.

We assess LLMs’ ability to estimate query performance based on their recognition of query complexity. This evaluation goes beyond simple syntactic analysis and requires an understanding of how different query components impact execution time. For instance, queries with deeply nested subqueries, multiple joins across large tables, or complex predicate conditions may be computationally expensive. If an LLM can identify such patterns and predict relative execution costs, it suggests a deeper *understanding* of SQL beyond just recognizing keywords and syntax.

Additionally, performance estimation is a fundamental aspect of SQL comprehension because it is directly relevant to practical applications, such as query optimization and workload management. Database administrators and automated systems use performance predictions to improve query execution plans, allocate resources efficiently, and prevent performance bottlenecks. Evaluating LLMs on this task helps determine whether they can develop an intuition for query complexity, similar to how experienced SQL users anticipate which queries might be slow or require optimization [62].

By testing LLMs on performance estimation, we measure their ability to reason about SQL execution beyond surface-level syntax. This assessment provides insights into whether LLMs can go beyond recognizing query structure to understanding how queries interact with database constraints, indexing strategies, and workload distribution.

1.1.4 Query Equivalence

Two SQL queries are considered equivalent if they produce identical results across all possible database instances, despite having different syntactic representations. Recognizing query equivalence is essential for improving database performance through query optimization techniques [8, 25] and enhancing user experience in query recommendation systems [67]. Query optimization benefits from identifying equivalent yet more efficient formulations, while recommendation systems can suggest simpler query alternatives that improve interpretability and execution speed.

To evaluate query equivalence, we use a dataset of labeled query pairs classified as either equivalent (positive) or non-equivalent (negative). Constructing such pairs is a non-trivial task, particularly for the negative cases. A naïve approach might involve selecting two random queries and labeling them as non-equivalent if they return different results. However, this oversimplifies the problem and primarily captures superficial differences, failing to test whether the model truly understands SQL query semantics. Instead, distinguishing non-equivalent queries in a meaningful way requires careful consideration of subtle differences in query logic, execution behavior, and result sets.

Our study examines whether LLMs can determine query equivalence, even when queries differ significantly in syntax but maintain the same semantics. This capability is crucial for various database-related tasks where recognizing equivalent queries can optimize processing and improve system efficiency. However, the complexity arises in cases where non-equivalence is not immediately obvious—two queries may appear similar but yield different results due to

Skill	Syntax Error	Missing Token	Q. Perf. Estimate	Q. Equiv.	Q. Explain.
Recognition	✓	✓			
Semantics				✓	✓
Context		✓	✓		✓
Coherence	✓		✓	✓	

Table 1.1: Skill-to-SQL task mapping

slight variations in filtering conditions, aggregation functions, or join constraints. Detecting such cases requires a deeper understanding of SQL execution, making this an advanced test of an LLM’s ability to comprehend structured query semantics.

By evaluating LLMs on this task, we assess whether they can go beyond surface-level syntactic recognition and develop a true understanding of SQL logic. This analysis provides insight into their ability to interpret query transformations, recognize redundant operations, and ultimately contribute to more efficient database management.

1.1.5 Query Explainability

We evaluate the ability of LLMs to explain SQL queries by generating descriptions of their outputs. This task is comparable to existing assessments in code and image understanding, where models generate code documentation [40] or image captions to measure their comprehension. Similarly, in the context of SQL, effective explanations require the model to interpret the query structure, understand the relationships between tables, and articulate the logical flow of operations.

Our evaluation covers a diverse range of SQL queries, including those with multiple tables, nested subqueries, and complex logical conditions. This comprehensive testing framework assesses whether LLMs can accurately parse and describe intricate database interactions, much like how they might translate or summarize code functions and visual elements in other domains. The ability to generate precise and meaningful SQL explanations is particularly valuable for applications such as automated documentation, query debugging, and user assistance systems.

By analyzing model performance across these varied query structures, we gain insight into the depth of their SQL *understanding* and their capacity to handle complex database queries. This evaluation also informs future refinements, improving the models’ precision in generating contextually accurate explanations and ensuring that their outputs are both useful and interpretable for SQL users.

Together, these tasks address fundamental challenges in SQL workflows and serve as key benchmarks for assessing LLMs’ SQL capabilities. While the selected tasks encompass a broad range of SQL applications, we acknowledge that this study does not aim to comprehensively evaluate LLMs across all possible SQL skills and tasks. Instead, it provides an in-depth experimental analysis of some of the most critical and foundational SQL-related tasks, offering valuable insights into the strengths and limitations of LLMs in structured query understanding.

1.2 Thesis Contributions and Structure

We present an experimental study evaluating the performance of major LLMs on core SQL tasks. Our study systematically investigates how well LLMs understand SQL by assessing their performance across multiple key tasks, from syntax error detection to query equivalence and performance estimation. The main contributions of this thesis are as follows:

- *SQL task-driven data benchmark.* Many of the evaluated SQL tasks require labeled data, which we generate by modifying raw queries from widely used SQL workloads, including the Sloan Digital Sky Survey (SDSS) [62], SQLShare [18], and the Join Order Benchmark [29]. For syntax error detection and missing token identification, we construct semi-synthetic datasets by randomly selecting queries from these workloads and injecting errors or removing tokens. Each task-specific dataset is generated by selecting appropriate error types, such as syntax violations or missing elements like keywords, table names, or column names.

For query performance estimation, we utilize the SDSS workload, which includes query execution logs. Queries are categorized based on runtime, with high-runtime queries representing computationally expensive operations.

For query equivalence tasks, we manually modify selected queries to create equivalent and non-equivalent pairs, ensuring that the modifications reflect realistic query transformations, such as rewriting nested queries using joins. Our benchmark dataset is publicly available.¹

- *Prompt-to-SQL task performance.* Effective prompt tuning is critical for obtaining consistent and reliable responses from LLMs. We experiment with various prompt designs, conducting small-scale trials using a subset of labeled data to identify the most effective prompt for each SQL task. However, LLM interactions involve more than just crafting prompts—processing their responses is a non-trivial challenge. To extract structured outputs from LLM-generated responses, we employ a combination of automated scripts and manual verification to ensure accuracy in labeling.
- *SQL task evaluation framework.* We systematically analyze the factors influencing LLM performance across SQL tasks by considering three key dimensions. First, we compare the performance of different LLMs on each SQL task to determine relative strengths and weaknesses. Second, we examine query workload properties, focusing on syntactic complexity, such as the number of tables, filtering conditions, nested subqueries, and query length. This analysis helps determine how structural aspects of queries impact LLM performance. Third, we evaluate performance variation across different task parameters, such as detecting specific types of syntax errors, missing tokens, or query transformations. For example, we analyze whether certain types of missing tokens (e.g., missing SELECT vs. missing table names) are more challenging for LLMs, and whether different forms of query equivalence or error detection pose varying degrees of difficulty. By examining LLM performance across these three dimensions—model comparison, query complexity, and task-specific variations—we provide a comprehensive evaluation of the factors influencing SQL task performance.

¹https://github.com/AnanyaRahaman/LLMs_SQL_Understanding

- *Extensive comparative evaluation.* Our experimental results indicate that GPT4 consistently achieves the highest performance across most SQL tasks, while no other model consistently ranks second. While many models perform well on binary classification tasks, such as identifying syntax errors or detecting missing tokens, their accuracy drops significantly in multi-class tasks, such as classifying the type of missing token or distinguishing different syntax errors.

Additionally, LLMs generally struggle with longer and more complex queries, particularly those involving logical reasoning or numerical computations, a trend consistent with prior findings [16, 9, 60]. This suggests that while LLMs can recognize and process SQL queries to some extent, their deeper semantic understanding remains limited.

Our framework is model-agnostic and designed to be easily extendable to accommodate newer LLMs or emerging tools. One of the key contributions of our work is the development of a benchmark suite that systematically evaluates SQL understanding through a diverse set of tasks. Since the benchmark and evaluation pipeline are modular and dataset-driven, they can be reused or adapted to assess any new language model. This ensures that as newer LLMs are introduced, their performance can be fairly and consistently compared against existing models, enabling long-term usability of our evaluation framework.

Our results demonstrate that LLMs perform well in tasks that require recognition and context awareness, such as syntax error detection and missing token identification. However, when tasks demand logical coherence and deeper semantic understanding, such as query equivalence and performance estimation, the models exhibit clear limitations. This indicates that while LLMs exhibit strong surface-level *understanding* of SQL, they struggle with more intricate reasoning and semantic consistency. These findings highlight the need for further advancements to enhance LLMs’ ability to process and comprehend structured query languages at a deeper level.

The thesis is structured as follows. In Chapter 3, we describe the query workloads used in the study, providing comprehensive statistics on the syntactic properties of the datasets. Chapter 4 outlines the experimental setup, detailing the SQL tasks, data generation process, LLM selection, and interaction strategies such as prompt tuning. Chapter 5 presents the experimental results and analysis. Chapter 2 reviews related work, and we conclude with a discussion of key findings and future research directions in Chapter 6.

Chapter 2

Related Work

Recent advancements in LLMs have significantly influenced various aspects of data management. These models are now widely used to streamline complex tasks such as data wrangling, entity matching, table manipulation, and text-to-SQL generation. By automating and optimizing these tasks, LLMs improve efficiency, accuracy, and accessibility in database management. In this chapter, we categorize the relevant literature into four main sections: (1) LLMs for data processing and entity matching, (2) LLMs for structured data manipulation and SQL interaction, (3) prompting techniques for improving LLM reasoning, and (4) evaluation and comparative analysis of LLM capabilities.

2.1 LLMs for Data Processing and Entity Matching

LLMs have been increasingly used for automating data processing tasks, including data wrangling, error detection, and structured data transformations. Narayan et al. [41] show that foundation models can automate tasks such as detecting inconsistencies, transforming data formats, and integrating diverse datasets. Their results indicate that LLMs not only improve the efficiency of data wrangling but also enhance accuracy, particularly in handling unstructured and ambiguous data. Similarly, Li et al. [31] propose an approach that leverages LLMs for structured data transformation, optimizing tasks like unit conversion and error detection while reducing computational costs.

In entity matching, LLMs have been applied to improve the identification of similar or duplicate records across different datasets. Zhang et al. [64] introduce AnyMatch, a zero-shot entity matching model that achieves competitive performance using a small, specialized LLM. This approach efficiently selects relevant data and performs comparably to larger models like GPT-4 while requiring fewer computational resources. Steiner et al. [54] explore the fine-tuning of LLMs for entity matching, demonstrating that fine-tuned models achieve improved accuracy but may struggle with cross-domain generalization. These studies highlight the potential of LLMs in automating and optimizing structured data processing tasks.

2.2 LLMs for SQL and Structured Data Processing

LLMs have also been applied to structured data tasks such as table manipulation and SQL query processing. Li et al. [30] introduce Table-GPT, a fine-tuned model designed for tasks such as data cleaning and table-based question answering. Their study finds that LLMs pre-trained on unstructured text perform poorly on structured tabular data, emphasizing the need for domain-specific fine-tuning.

A particularly promising application of LLMs is in text-to-SQL translation, where natural language queries are converted into executable SQL statements. Surveys by Hong et al. [19] and Gao et al. [17] analyze the challenges in text-to-SQL generation, noting that while LLMs perform well on simple queries, they struggle with complex query structures involving nested subqueries, multiple joins, and aggregation functions. This suggests that while LLMs can aid SQL generation, further improvements are needed to handle the intricacies of real-world database queries.

Additionally, LLMs have been used in query recommendation and interactive database querying. Eirinaki et al. [13] propose QueRIE, a system that enhances database exploration by recommending queries based on user interactions. Similarly, Khoussainova et al. [23] develop SnipSuggest, which provides real-time, context-aware SQL autocompletion, reducing query formulation time and minimizing errors. Other works focus on query suggestion and interactive SQL support, including interactive query formulation systems [15, 21] and recommendation-based query assistance tools [7, 53, 51, 56].

2.3 Prompting Techniques for Enhancing LLM Reasoning

Structured prompting has emerged as an important technique to improve LLM accuracy in complex reasoning tasks. Arora et al. [4] propose a structured prompting framework that enhances model responses in tasks such as question answering and problem-solving. Liu et al. [32] review various prompting strategies, demonstrating how pre-trained models adapt to different tasks through well-crafted prompts.

Shao et al. [50] introduce synthetic prompting, where models generate chain-of-thought reasoning steps to improve performance in multi-step tasks. This technique helps break down complex problems into simpler sub-tasks, improving accuracy and interpretability. Lu et al. [35] address the issue of prompt order sensitivity in few-shot learning, proposing a methodology to optimize prompt sequences for more stable results. Additionally, Zhao et al. [65] explore pre-task calibration, a method for refining model performance before applying them to few-shot learning scenarios. Their approach adjusts model parameters based on small task-specific datasets, improving accuracy and reducing inconsistencies.

2.4 Evaluating and Comparing LLM Capabilities

A key area of research is the evaluation and comparison of LLM capabilities across different tasks and domains. Akter et al. [1] analyze the linguistic capabilities of Gemini, comparing it to models like GPT-3 and BERT. Their findings highlight Gemini's strengths in machine

translation and sentiment analysis, positioning it as a competitive model for global communication. Similarly, Lee et al. [28] compare Gemini Pro and GPT-4V in educational applications, demonstrating GPT-4V's superior performance in tasks such as automated essay scoring and personalized learning.

Further evaluations of LLMs focus on their generalizability, trustworthiness, and causal reasoning across different modalities. Lu et al. [34] assess GPT-4 and Gemini's capabilities in text, image, audio, and video processing, identifying limitations in multi-modal reasoning. Rosenfeld and Lazebnik [48] investigate attribution in LLM-generated text, developing methodologies to trace model-generated outputs for accountability purposes.

Comparative evaluation tools have also been developed to facilitate the systematic analysis of LLMs. Kahng et al. [22] introduce the LLM Comparator, a visual analytics tool that enables intuitive comparison of LLM performance across various conditions. Chang et al. [6] provide a comprehensive survey on LLM evaluation methodologies, identifying gaps in current assessment frameworks and calling for more holistic evaluation criteria that consider both technical performance and ethical implications.

LLMs have demonstrated significant progress in automating data processing, improving SQL interaction, and enhancing structured prompting techniques. Research has shown that while LLMs perform well in structured data tasks such as entity matching, table manipulation, and text-to-SQL translation, they still face challenges in handling complex query structures and multi-step reasoning. Furthermore, structured prompting methods have improved model performance in various applications, but limitations such as sensitivity to prompt ordering and cross-domain generalization remain open research areas.

The comparative analysis of LLMs highlights both their strengths and weaknesses across different modalities, emphasizing the need for continued improvements in generalizability, explainability, and causal reasoning. While LLMs offer promising advancements in data management, further research is necessary to enhance their scalability, efficiency, and adaptability in diverse real-world applications.

Chapter 3

Query-Workloads

A query workload, which is often just referred to as a workload, consists of a set of SQL queries that are run against a database system. This collection of queries is specifically designed to mimic the typical interactions that might occur under real-world conditions. The primary purpose of deploying such workloads is to evaluate how a database performs under different scenarios. This evaluation helps in assessing the performance of the database, identifying potential bottlenecks, and determining areas that might benefit from optimization to enhance overall efficiency and effectiveness.

In our experimental study, we utilize four distinct workloads to examine our database system. Each workload has been carefully compiled to represent different user behavior patterns and query types that are expected in a typical usage environment. We provide both an overview and a detailed analysis of these workloads. This includes insights into the types of queries included, their complexity, the frequency of their execution, and how they interact with the database schema. Such comprehensive analysis aids in understanding how well the database can handle various operational demands and assists in making informed decisions about where improvements can be made to optimize performance.

- *The Sloan Digital Sky Survey (SDSS) dataset [62]*. SDSS consists of a relational database filled with comprehensive data from an extensive astronomical survey, which includes detailed visual and spectral information of the cosmos. The database is interacted with through a specially designed SQL query workload, which facilitates the retrieval and analysis of this detailed astronomical data. These queries are intricately crafted to handle complex information, making it possible for researchers and astronomers to perform precise and scientifically meaningful analyses.

Over two decades, the SDSS workload has accumulated millions of queries, representing a wide spectrum of data inquiries and analytical processes. In our study, we specifically examine the subset of queries recorded in 2023, providing insight into how the database manages current scientific data demands. This analysis helps us evaluate the efficiency of the SDSS database, ensuring it continues to serve the advanced and evolving needs of the astronomical community effectively.

- *SQLShare [18]*. SQLShare is an open data platform specifically designed to facilitate easier data sharing and querying for users of all skill levels. It aims to democratize data

Workload	Number of Queries		Query Type		Aggregate		NestLvl	
	Original	Sampled	SELECT	CREATE	Yes	No	0	1
SDSS	5,081,188	285	Fig 3.1a		21	264	Fig 3.1e	
SQLShare	9,623	250	Fig 3.2a		59	192	Fig 3.2e	
Join-Order	157	157	113	44	119	38	-	-
Spider	4, 486	200	200	0	96	104	185	15

Table 3.1: Workload statistics overview

access by allowing users to easily execute a variety of SQL queries, ranging from simple data retrieval operations to more complex data manipulation tasks. This broad range of capabilities makes SQLShare a versatile tool for both novice and experienced users who need to interact with data in meaningful ways.

Distinctively, the SQLShare workload differs from others in that it includes queries that operate across multiple databases. Each of these databases has its own unique schema, introducing a level of complexity that requires users to be adaptable and knowledgeable about different data architectures. This aspect of SQLShare is particularly valuable for testing the flexibility and resilience of SQL queries in a multi-database environment, providing a realistic and challenging landscape for data management and analysis. This environment not only tests the robustness of the SQL queries but also enhances user skills in managing diverse data structures under varying scenarios.

- *Join-Order [29]*. The Join-Order Benchmark is a synthetic workload developed to test the efficiency of database systems in optimizing join queries. This benchmark is crucial for assessing how well a database’s optimizer can handle SQL queries that involve multiple join operations, which are fundamental in relational databases for merging data across tables. The Join-Order Benchmark includes a series of complex SQL queries designed to challenge the optimizer’s ability to determine the most efficient join order, directly impacting the speed and resource efficiency of the database. This benchmark thus serves as a key tool for evaluating and improving the performance of database management systems.
- *Spider [63]*. Spider is a large-scale, complex, cross-domain Text-to-SQL benchmark used to evaluate a model’s natural language understanding, and SQL generation capabilities. It includes a diverse array of databases with different schemas to test a model’s ability to generalize across various data environments, making it crucial for applications requiring user interaction with database systems via natural language, such as chatbots or virtual assistants. Spider is extensively used in NLP to benchmark the translation of natural language queries into SQL. In our use, the Spider dataset is specifically employed for query explanation, allowing for in-depth analysis of how SQL queries generated from natural language are understood and interpreted, while other workloads address broader tasks.

All our workloads, with the exception of Join-Order, contain a large number of queries; making it impractical to utilize all queries in our study. Therefore, we created smaller datasets by sampling a limited number of queries.

Table 3.1 shows the “original” number of queries in the workloads, and “sampled” shows the sampled number of queries we use in our experiments. We describe the dataset generation process in Section 4.2. Next, we examine the syntactic properties of our sampled queries, to provide context to interpret our experimental results. (which will assist in interpreting the results in later sections.) Henceforth, we will use SDSS, SQLShare, Join-Order, and Spider to refer to the datasets created from the sampled queries of the original workloads.

While the selected workloads are widely used in the literature, they may not fully capture the characteristics of SQL tasks in specialized domains, which often exhibit unique and domain-specific features. In our discussions, we emphasize the specific workload properties used in our thesis, and their influence on LLM performance. We study LLM performance on domain-specific, specialized, SQL workloads as an avenue of future research.

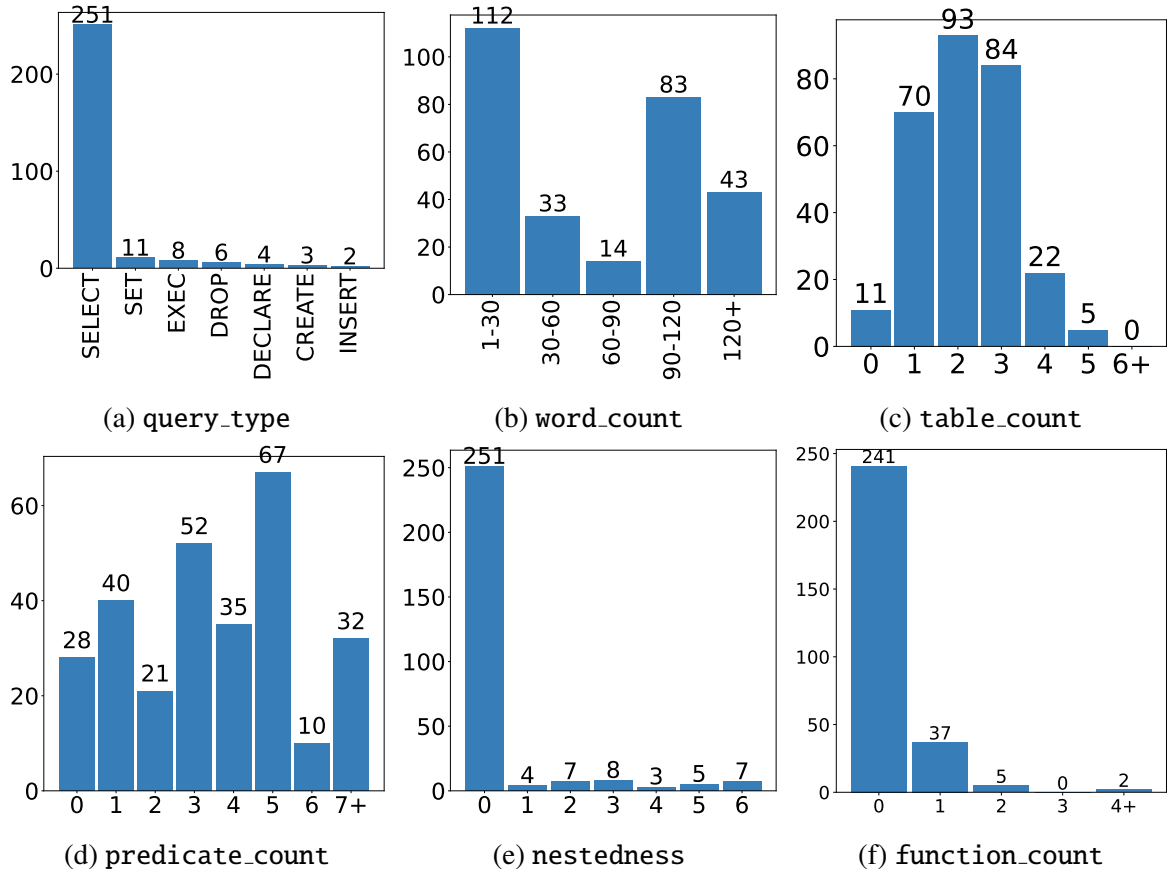


Figure 3.1: SDSS Statistics

3.1 Syntactic Properties of SQL Queries

For each SQL query, we assess the following properties:

- `char_count` and `word_count`, respectively, refer to the number of characters and the number of words in the query.

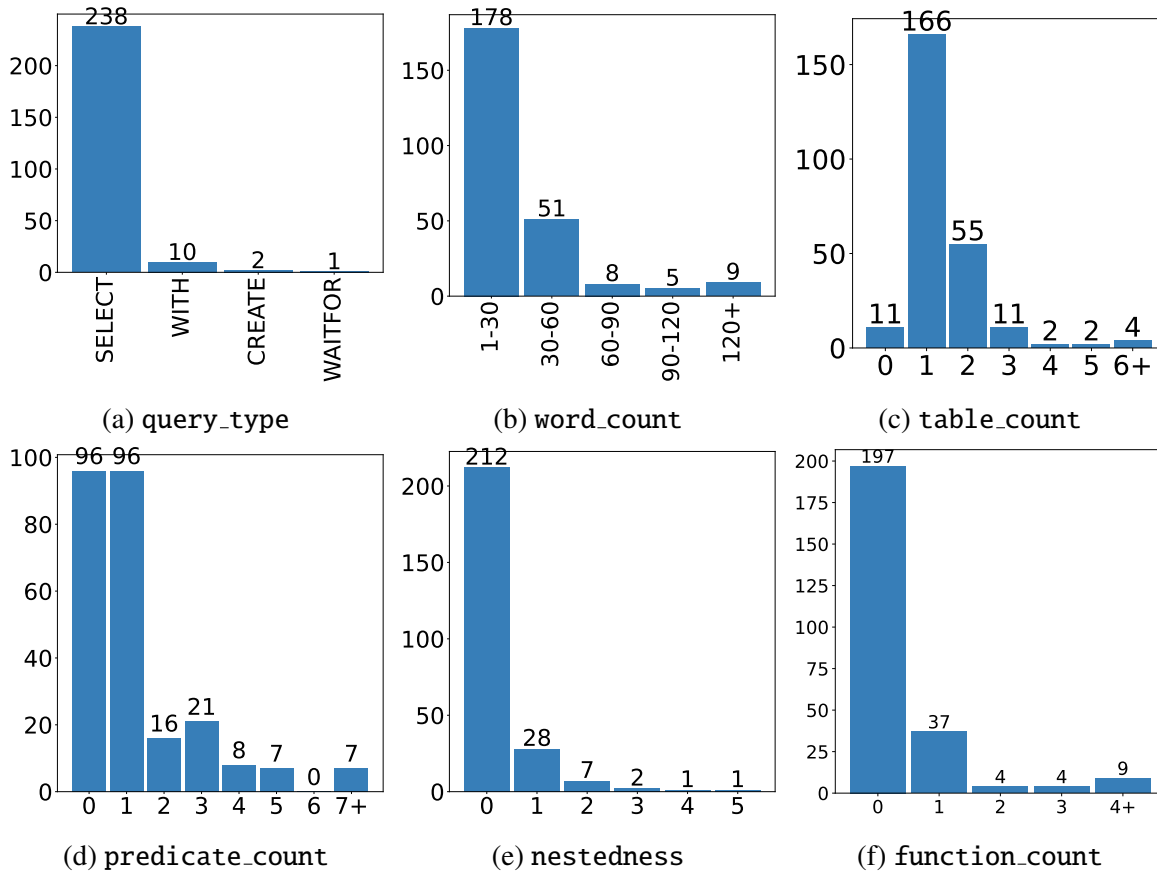


Figure 3.2: SQLShare Statistics

- `query_type` refers to the type of the query, e.g., `SELECT`, `UPDATE`, and `CREATE`.
- `table_count` and `join_count` refer to the number of distinct tables referenced in the query and the total number of joins, respectively. Joins include both explicit joins (using join keywords such as `INNER JOIN`) and implicit joins (tables in the `FROM` clause with join conditions).
- `column_count` refers the number of distinct columns used or referenced in the `SELECT` clause of the query.
- `function_count` refers to the total number of functions in the query, including built-in (like `min`, `avg`) and user-defined functions. `predicate_count` is the number of conditions specified in the `WHERE` clause.
- `nestedness` is the nested depth of subqueries within the query.
- `aggregate` refers to whether the query uses aggregate functions.

Table 3.1 provides a statistical overview of all four workloads, including the number of `SELECT` and `CREATE` queries and a breakdown of aggregate vs. simple queries. Figures 3.1-3.6 illustrate additional properties. Each figure is a histogram showing query counts on the

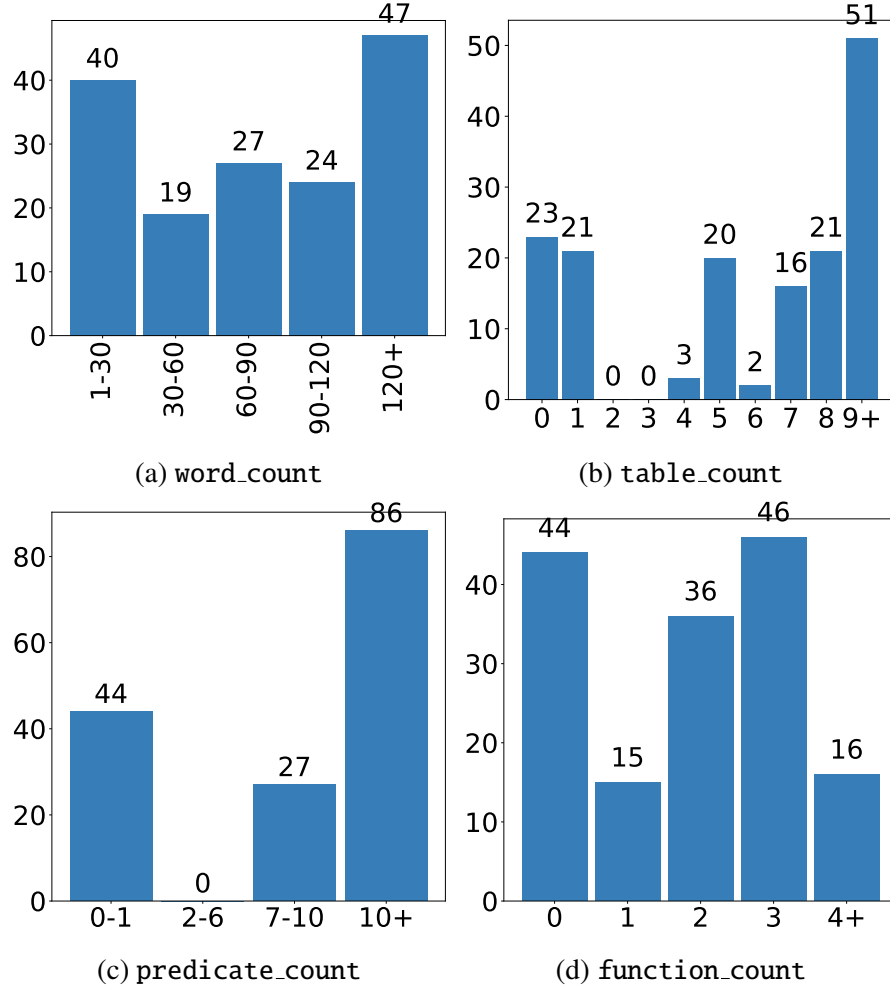


Figure 3.3: Join Order Statistics

y-axis and query properties on the x -axis, where the x -values represent a range of properties. For example, Figure 3.1b shows the number of queries (y-axis) across different ranges of query lengths (word_count) in SDSS. The figures highlight that SDSS and SQLShare contain more complex queries, with multiple tables, high level of Nestedness and a wider variety of predicates. Moreover, these two workloads have variety of query_type, function_count and column_count. In contrast, Join-Order has simpler, less nested queries. For query length (word_count), SDSS and Join-Order have longer queries compared to SQLShare and Spider.

As pairs of properties may exhibit strong correlations, leading to redundancy and inefficiency, we examine the correlations between pairwise query properties using Pearson coefficients [45], and we use a threshold of 0.7 to indicate strong correlation. Figure 3.7 show the following observations:

- char_count and word_count are highly correlated, as longer queries generally contain more words.
- table_count and join_count are also highly correlated since queries with more tables

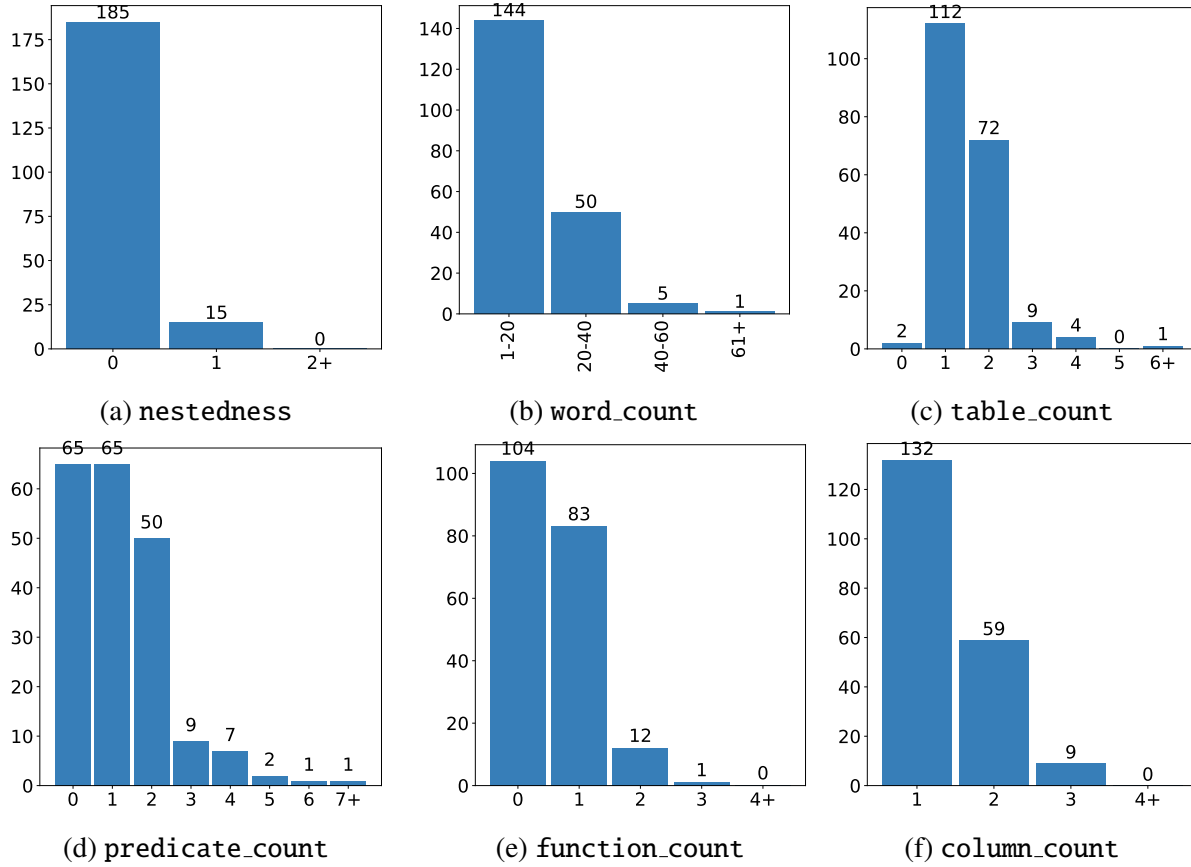


Figure 3.4: Statistics for Spider workload

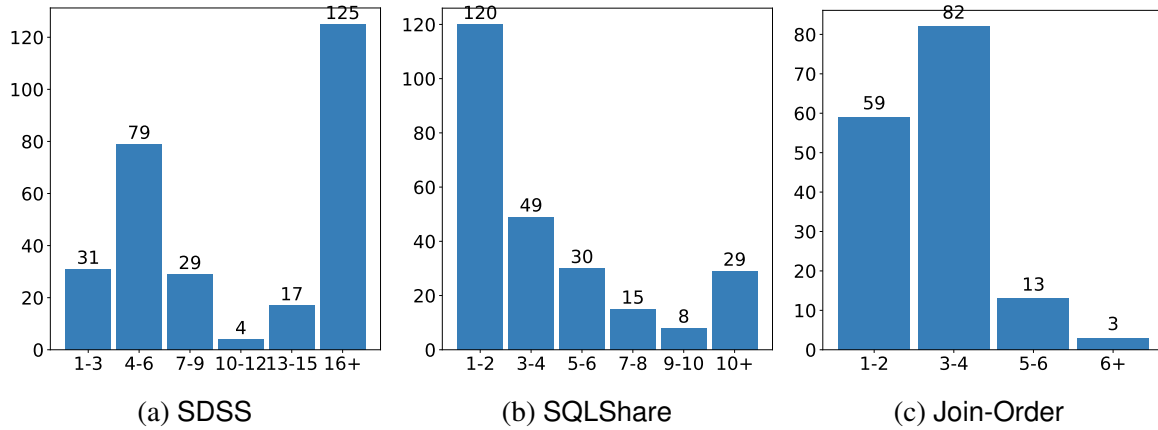
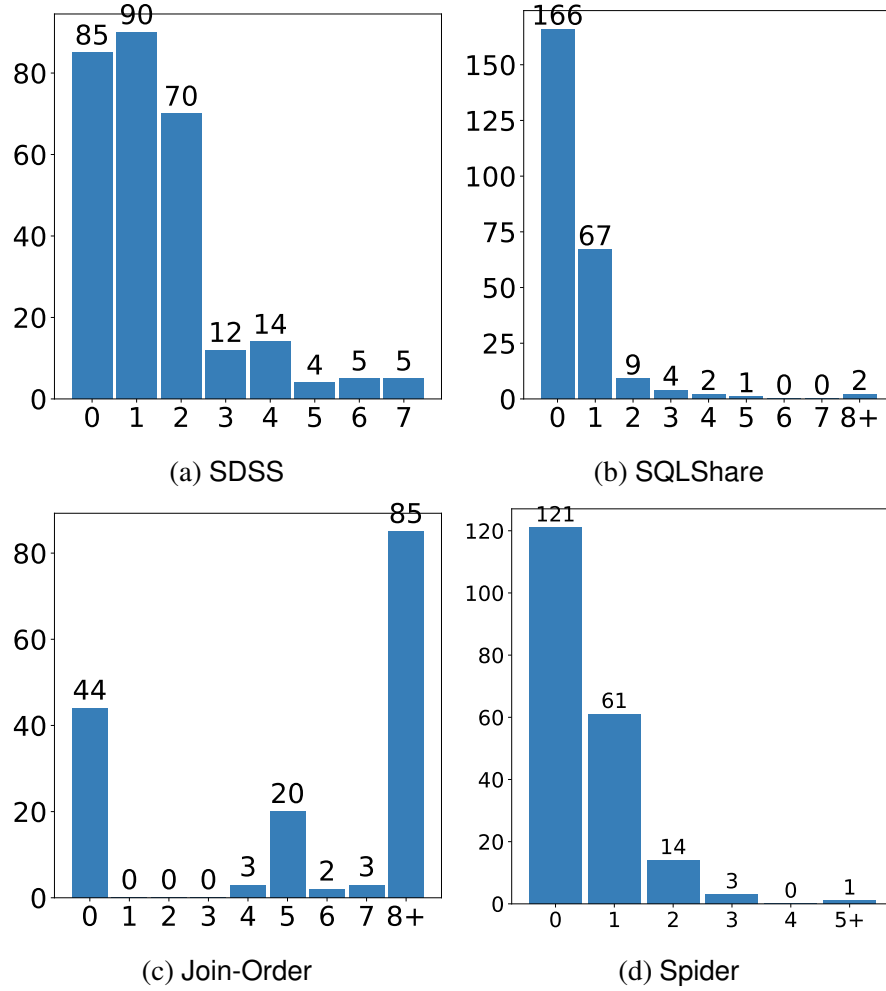


Figure 3.5: column_count for SDSS, SQLShare and Join-Order

usually involve more joins, a common pattern in multi-table SQL queries.

We consider correlations unique to specific workloads:

- In SDSS, `column_count` and `char_count` are strongly correlated as longer queries often involve selecting more columns or adding conditions. Additionally, `nestedness` and

Figure 3.6: `join_count` for all workloads

`join_count` are correlated, as deeply nested queries tend to include multiple joins.

- SQLShare and Join-Order exhibit a high correlation between `function_count` and `predicate_count` due to the frequent use of functions in conditions.
- In Join-Order, `char_count`, and `word_count` are correlated with `table_count` and `join_count`, indicating longer queries involve more joins and tables.
- Spider additionally exhibits correlation between `predicate_count` and (`char_count`, `word_count`, `join_count`, `table_count`) are all highly correlated, reflecting the simplicity and lack of diversity in the queries.

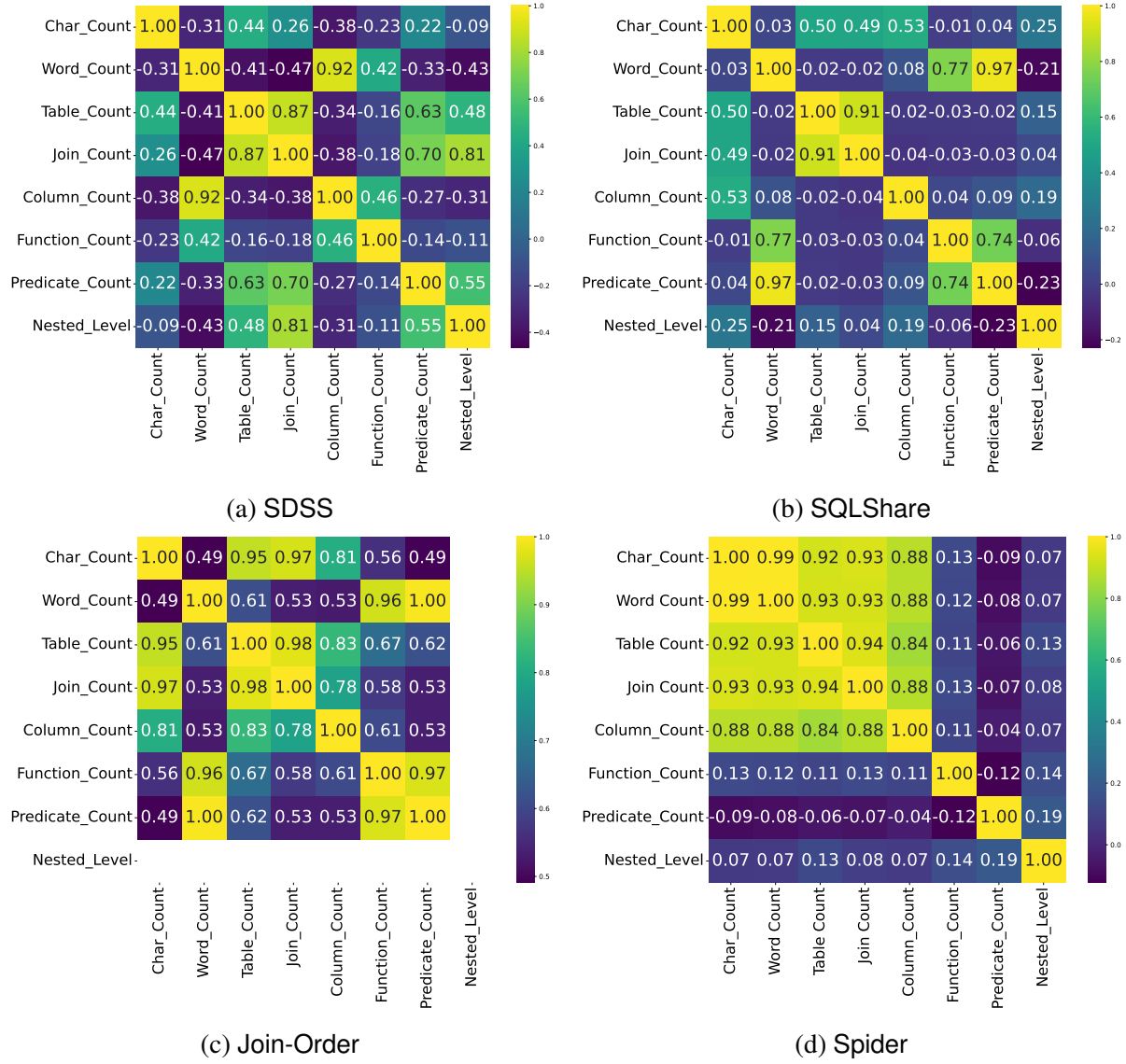


Figure 3.7: Pairwise correlations between query properties for each workload

Chapter 4

Experimental Framework

We introduce our SQL tasks in Section 4.1, and our data preparation steps to inject errors, missing tokens, and derive equivalent and non-equivalent queries in Section 4.2. We then give an overview of the evaluated LLMs (Section 4.3), and how we prompt and respond to the LLMs in Section 4.4.

4.1 SQL Tasks

To assess SQL “understanding” in LLMs, we conducted experiments on several tasks involving SQL queries.

4.1.1 Binary Tasks

We begin with binary classification tasks that identify syntactic errors, missing tokens, and query equivalence.

syntax_error. We evaluate the LLM’s ability to identify the presence of a syntax error. We study six types of errors as described below. Listing 4.1 shows sample errors for each type.

- **aggr-attr.** Aggregate functions are used without properly grouping non-aggregated columns.
- **aggr-having.** Misusing the HAVING clause to filter non-aggregated columns instead of using WHERE.
- **nested-mismatch.** The inner query in a nested query returns multiple rows, which is not correctly handled in the outer query.
- **condition-mismatch.** Operations with incompatible data types, e.g., comparing numeric columns to strings.
- **alias-undefined.** An alias is used in a query but is not defined.
- **alias-ambiguous.** The same column appears in multiple tables, but its usage in a query does not specify the table reference.

```

-- Q1: Aggregation without GROUP BY (aggr-attr)
SELECT plate,mjd,COUNT(*), AVG(z)
FROM SpecObj WHERE z > 0.5;
-- Q2: Incorrect Use of HAVING (aggr-having)
SELECT plate,COUNT(*) AS NumSpectra
FROM SpecObj GROUP BY plate HAVING z > 0.5;
-- Q3: Type mismatch in subquery (nested-mismatch)
SELECT p.ra,p.dec,s.z
FROM PhotoObj AS p JOIN SpecObj AS s
ON s.bestobjid = (SELECT bestobjid FROM SpecObj);
-- Q4: Type mismatch in condition (condition-mismatch)
SELECT plate,mjd,fiberid FROM SpecObj WHERE z = 'high';
-- Q5: Undefined alias (alias-undefined)
SELECT s.plate,s.mjd,z
FROM SpecObj AS s JOIN PhotoObj AS p
ON s.bestobjid = photoobj.bestobjid;
-- Q6: Ambiguous alias (alias-ambiguous)
SELECT plate,fid FROM SpecObj AS s JOIN PhotoObj AS p
ON s.bestobjid = p.bestobjid WHERE bestobjid > 1000;

```

Listing 4.1: SQL syntax error examples

miss_token. We evaluate whether an LLM can identify missing tokens in a SQL query. While related to `syntax_error`, this task is treated separately due to its practical importance. We consider six token types: `keyword`, `table`, `column`, `value`, `alias`, and `predicate` (comparisons). Missing keywords include essential clauses like `SELECT`, `WHERE`, or `JOIN`, which define query structure. Missing tables refer to absent table names in the `FROM` clause, causing execution ambiguity. Missing columns involve omitted column names in `SELECT` or conditions, critical for defining output or filters. Missing values include absent literals in conditions (e.g., `WHERE age ≤`), leaving comparisons incomplete. Missing aliases result in ambiguity when table or subquery aliases are omitted. Missing predicates involve incomplete conditions, such as “age 50” where the comparison operator, e.g., `≤`, is missing.

query_equiv. Determines whether two SQL queries are equivalent, i.e., whether they have the same schema and produce the same results. We study ten types of equivalences and eight types of non-equivalences. Listings 4.2 shows examples of these types using the SDSS workload.

- **swap-subqueries.** Swapping inner and outer sub-queries in nested queries.
- **Changing Join Order.** Altering the order of tables in the join clause.
- **cte.** Rewriting a query using common table expressions (CTEs), a temporary result set defined using `WITH`, which simplifies complex queries and is referenced within the main query.
- **reorder-conditions.** Re-arranging the order of conditions in a `WHERE` clause.
- **Converting Joins.** Changing the style of join syntax from explicit joins, which use the `JOIN` keyword, to implicit joins, where the join condition is specified in the `WHERE` clause.
- **Using Subqueries.** Replacing a join operation and direct conditions with subqueries.

- **Using UNION or INTERSECT ALL.** Using set operations and their implications to make equivalent queries.
- **Using EXISTS.** Using EXISTS logical operator within subqueries that can replicate the effects of joins or complex conditions.
- **Using CASE Statements.** Using conditional logic (CASE and WHEN) within queries to produce equivalent queries.
- **Simplification and Direct Application.** Simplifying complex queries while maintaining the same logical conditions and outputs.

We study eight types of non-equivalent transformations:

- **agg-function.** Modifying an aggregate function, e.g., updating to SUM from AVG.
- **change-join-condition.** Modifying the type of join, such as switching from an INNER JOIN to a LEFT JOIN.
- **logical-conditions.** Altering logical operators, such as changing AND to OR.
- **value-change.** Updating a filtering condition, e.g., altering the comparison value.
- **Operator-change.** Updating an Operator, e.g., altering the $<$ (less than) operator with $>$ (greater than) operator.
- **Modifying Grouping Criteria.** Changing grouping fields and observes its impact on aggregation.
- **Changing Sorting and Limiting.** Varying ordering and pagination to see its effect on the query output.
- **Modifying Order By Criteria.** Modifying Order By criteria to test the order significance.
- **Ambiguous Column References.** Introduces errors in column references to evaluate error detection capabilities.
- **Change Table Name.** Introducing errors in table names.

```
-- Q7: swap-subqueries (Equivalent)
SELECT objID FROM PhotoObj
WHERE objID IN ( SELECT objID
FROM PhotoTag WHERE u > 18);
-- Equivalent Query:
SELECT objID FROM PhotoTag
WHERE u > 18 AND objID IN (
SELECT objID FROM PhotoObj)
-- Q8: Changing Join Order (Equivalent)
SELECT p.objID, s.specObjID
FROM PhotoObj AS p, SpecObj AS s
```

```

WHERE p.objID = s.objID AND p.u < 19;
-- Equivalent Query:
SELECT p.objID, s.specObjID
FROM SpecObj AS s, PhotoObj AS p
WHERE p.objID = s.objID AND p.u < 19;
-- Q9: cte (Equivalent)
SELECT plate,mjd FROM SpecObj WHERE z > 0.5;
-- Equivalent Query:
WITH HighRedshift AS
    (SELECT plate,mjd FROM SpecObj WHERE z > 0.5)
SELECT plate,mjd FROM HighRedshift;
-- Q10: reorder-conditions (Equivalent)
SELECT * FROM SpecObj WHERE plate = 1000 AND mjd > 55000;
-- Equivalent Query:
SELECT * FROM SpecObj WHERE mjd > 55000 AND plate = 1000;
-- Q11: Converting Joins ("Explicit_Implicit_Joins")(Equivalent)
SELECT p.objID, s.specObjID
FROM PhotoObj p, SpecObj s
WHERE p.objID = s.objID;
-- Equivalent Query
SELECT p.objID, s.specObjID
FROM PhotoObj p
JOIN SpecObj s ON p.objID = s.objID;
-- Q12: Using Subqueries ("Join_Nested","Condition_Nested")(Equivalent)
Join to Nested Example:
SELECT p.objID, p.ra, p.dec PhotoObj p
JOIN Field f ON p.fieldID = f.fieldID
WHERE f.ra > 250;
-- Equivalent Query
SELECT p.objID, p.ra, p.dec FROM PhotoObj p
WHERE p.fieldID IN ( SELECT fieldID
FROM Field WHERE ra > 250);
Condition to Nested Example:
SELECT objID, ra, dec FROM PhotoObj
WHERE fieldID = 100;
-- Equivalent Query
SELECT objID, ra, dec FROM PhotoObj
WHERE fieldID = (SELECT fieldID FROM Field WHERE fieldID = 100);
-- Q13: Using UNION or INTERSECT ALL ("Condition_IntersectUnion")
SELECT objIDFROM PhotoObj
WHERE ra < 180 OR ra > 185;
-- Equivalent Query
SELECT objID FROM PhotoObj
WHERE ra < 180 UNION
SELECT objID FROM PhotoObj
WHERE ra > 185;
-- Q14: Using EXISTS ("Condition_EXISTS")
SELECT p.objID FROM PhotoObj p
JOIN SpecObj s ON p.objID = s.objID
WHERE s.z > 0.1;
-- Equivalent Query
SELECT objID FROM PhotoObj p
WHERE EXISTS (SELECT 1
FROM SpecObj s

```



```

WHERE p.objID = s.objID AND s.z > 0.1);
-- Q15: Using CASE Statements ("Case_Statement")
SELECT p.objID, p.type, o.priority_modifier, p.magnitude *
(1 - o.priority_modifier) AS adjusted_magnitude FROM PhotoObj p
LEFT JOIN ObjectTypes o
ON p.type = o.type;
-- Equivalent Query
SELECT objID, type, magnitude,
CASE
WHEN type = 'STAR' THEN magnitude * 0.95 -- Slightly lower priority for
stars
WHEN type = 'GALAXY' THEN magnitude * 0.90 -- Medium priority for galaxies
WHEN type = 'QSO' THEN magnitude * 0.85 -- High priority for quasars
ELSE magnitude -- No adjustment for unknown types
END AS adjusted_magnitude FROM PhotoObj;
-- Q16: Simplification and Direct Application ("Simplification")
SELECT MAX(redshift) AS max_redshift,
MIN(redshift) AS min_redshift,
AVG(redshift) AS avg_redshift,
COUNT(*) AS total_count
FROM (SELECT objID, redshift
FROM SpecObj WHERE class = 'GALAXY'
AND redshift > 0.1 AND redshift < 0.3
)x ORDER BY redshift DESC;
-- Equivalent Query
SELECT MAX(redshift) AS max_redshift,
MIN(redshift) AS min_redshift,
AVG(redshift) AS avg_redshift,
COUNT(*) AS total_count
FROM SpecObj WHERE class = 'GALAXY'
AND redshift > 0.1 AND redshift < 0.3;
-- Q17: agg-function (Non-Equivalent)
SELECT plate,AVG(z) FROM SpecObj GROUP BY plate;
-- Non-Equivalent Query:
SELECT plate,SUM(z) FROM SpecObj GROUP BY plate;
-- Q18: change-join-condition (Non-Equivalent)
SELECT s.plate,s.mjd FROM SpecObj AS s
JOIN PhotoObj AS p ON s.bestobjid = p.objid;
-- Non-Equivalent Query:
SELECT s.plate,s.mjd FROM SpecObj AS s
LEFT JOIN PhotoObj AS p ON s.bestobjid = p.objid;
-- Q19: logical-conditions (Non-Equivalent)
SELECT plate,mjd,fiberid
FROM SpecObj WHERE z > 0.5 AND ra > 180;
-- Non-Equivalent Query:
SELECT plate,mjd,fiberid
FROM SpecObj WHERE z > 0.5 OR ra > 180;
-- Q20: value-change (Non-Equivalent)
SELECT plate, mjd, fiberid FROM SpecObj WHERE z > 0.5;
-- Non-Equivalent Query:
SELECT plate,mjd,fiberid FROM SpecObj WHERE z > 5;
-- Q21: Operator Change(Non-Equivalent)
SELECT objID FROM PhotoObj WHERE mag_u > 18;
-- Non-equivalent Query

```

```

SELECT objID FROM PhotoObj WHERE mag_u < 18;
-- Q22: Modifying Grouping Criteria ("Group_by_Criteria")
SELECT fieldID, COUNT(*) FROM PhotoObj GROUP BY fieldID;
-- Non-equivalent Query
SELECT type, fieldID, COUNT(*) FROM PhotoObj GROUP BY type, fieldID;
-- Q23: Changing Sorting and Limiting ("Change_Sorting_Limiting")
SELECT objID, ra FROM PhotoObj ORDER BY ra DESC LIMIT 10;
-- Non-equivalent Query
SELECT objID, ra FROM PhotoObj ORDER BY ra ASC LIMIT 10;
-- Q24: Modifying Order By Criteria ("Order_by_Column")
SELECT objID, ra FROM PhotoObj ORDER BY ra;
-- Non-equivalent Query
SELECT objID, dec FROM PhotoObj ORDER BY dec;
-- Q25: Ambiguous Column References
SELECT objID, ra, dec FROM PhotoObj
FROM Employees;
-- Non Equivalent Query
SELECT objID, ra FROM PhotoObj
FROM Employees;
-- Q26: Change Table Name
SELECT objID, ra FROM PhotoObj;
-- Non Equivalent Query
SELECT objID, ra FROM PhotoObject;

```

Listing 4.2: Examples of SQL equivalence and non-equivalence

performance_pred. We evaluate the model’s ability to predict query runtime performance. Only the SDSS workload contains ground truth query execution times. Figure 4.1 shows a clear separation between short running (low-cost) vs. long running queries (costly), which we pose as a binary classification task, and consider costly queries as the positive class.

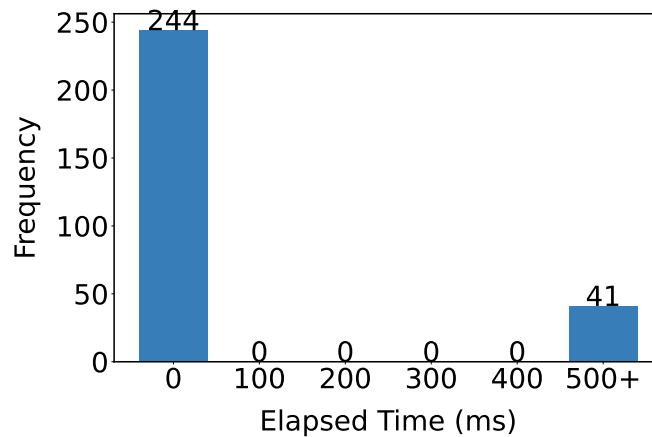


Figure 4.1: Elapsed time of sampled SDSS queries

4.1.2 Multi-Class Tasks

We are expanding our evaluation of LLMs by moving from binary to multi-class tasks, requiring LLMs to classify types of `syntax_error`, `miss_token`, and `query_equiv`. This involves identifying not just the presence of an error or missing element but specifying its nature—such

as the type of syntax error or the specific token missing—and the type of query equivalence observed. Additionally, we assess the model’s ability to precisely locate where a missing token should be in the query. This more detailed probing enhances the diagnostic capabilities of LLMs, allowing them to provide deeper insights into SQL query structure and errors, which is crucial for developing advanced query debugging tools.

4.1.3 Query Explanation

This task (**query_exp**) explains what a SQL query does. It is the reverse of the text-to-SQL task, where existing benchmarks for text-to-SQL, such as WikiSQL [66], provide natural language descriptions of queries. However, many of these benchmarks contain relatively simple SQL queries compared to the more complex workloads in SDSS and SQLShare. Thus, we chose the Spider dataset [63] that includes more complex queries, and we further sampled longer and more complex queries.

Our analysis is qualitative rather than quantitative. We manually review the LLM generated explanations, and compare them with the ground truth descriptions provided in the workload. Our goal is to analyze and discuss when and why models fail to provide accurate, meaningful explanations (Section 5.5). While this task does not strictly require existing explanations, we use Spider’s explanations to help with validation, and to streamline the evaluation process.

We deliberately chose not to rely on conventional text similarity metrics such as BLEU or ROUGE for the query explanation task, as these measures primarily focus on surface-level lexical overlap rather than deep semantic understanding. In the context of SQL, small lexical changes can represent significant semantic shifts—such as replacing MAX with MIN, which fundamentally alters the query’s intent. However, metrics like BLEU tend to assign high similarity scores even when such critical changes occur, thereby failing to capture meaningful discrepancies between the ground truth explanation (e.g., from the Spider dataset) and the model-generated explanation. In our preliminary experiments using BLEU, we observed exactly this limitation: semantically incorrect explanations often received unjustifiably high scores. Therefore, we argue that evaluating SQL explanation quality requires a more structured or semantics-aware approach that goes beyond traditional NLP similarity metrics.

4.2 Data Preparation and Label Generation

The original workloads do not include labeled data and often contain more queries than we can use. Below, We describe how task-specific labels are generated for each dataset for the SQL tasks.

syntax_error and **miss_token**. We generate semi-synthetic datasets by randomly selecting queries from each workload and injecting errors. We inject errors using GPT4 by providing the proper instructions for each Type. Then we manually check to ensure that the queries contain the specific errors we intended to inject. For **syntax_error**, we randomly select the type of error to inject, including cases with no error. We create binary labels to indicate the presence or absence of an error, and the error type for the multi-class (**syntax_error_type**) task. For error-free queries, we simply copy the original queries from the workloads, and

in `syntax_error_type`, we define these as 'NO ERROR'. After that, we manually checked everything to ensure correctness.

For `miss_token`, we followed a similar approach but focused on randomly removing a specific token type from the query to create a missing token scenario. We create binary labels to indicate the presence or absence of a missing token, and the error type for the multi-class (missing token type) task. Similar to `syntax_error_type`, we write 'No Missing' if the query is error-free. For `miss_token_loc`, we count the word from where the `miss_token` is missing and then note the word count position; we also write 0 if the query is error-free. Thus, for the `miss_token` task, we have binary labels: `miss_token_type` and `miss_token_loc`. Following that, we manually verified the accuracy of all elements. These datasets were created using the SDSS, SQLShare, and Join-Order workloads.

query_equiv. This task requires query pairs. To create a meaningful binary classification task, we need both equivalent and non-equivalent pairs. We generated these pairs using SDSS, SQLShare, and Join-Order, selecting random SELECT queries and manually modifying them to create equivalent queries. We randomly select the equivalence types, which specify the transformation for generating equivalent queries.

Generating non-equivalent pairs is also non-trivial, as they must be similar enough to make the task challenging. We use the predefined transformations that we explained. The decision to create equivalent or non-equivalent pairs, as well as the type of modification, was made randomly. Thus, each query pair not only has a label for equivalent or non-equivalent but also has a type referring to how it is generated, which is crucial for `query_equiv_type` and also our analysis.

performance_pred. For this task, we used only SDSS, which includes runtime data. We randomly selected 285 queries and analyzed their runtime to classify each as either high or low runtime, serving as a proxy for computational expense. Queries running longer than 500 ms are high cost, otherwise, they are low cost. We select this threshold based on our observations from Figure 4.1.

Clarifications on Data Preparation. While parts of the process, like error injection and query transformation, were automated, manual oversight ensured meaningful query modifications and accurate labels. This balance between automation and human intervention was crucial for maintaining data integrity and relevance. For instance, generating non-equivalent pairs required careful changes to preserve similarity while introducing functional differences, ensuring realistic datasets for effective evaluation. This involved strategically altering query structures, modifying join conditions, or tweaking aggregation logic while preserving the overall format. Additionally, meticulous validation steps were taken to confirm that introduced variations aligned with real-world query patterns, preventing the creation of misleading or impractical examples.

While the errors in our semi-synthetic dataset are not directly sourced from real-world query logs, they are carefully designed to reflect the types of mistakes frequently observed in practical settings. Our error injection strategies are grounded in known SQL error patterns—such as missing tokens, incorrect clause order, or misused aggregate functions—commonly encountered by users. In fact, in related real-world applications like SQL auto-completion and intelligent query assistance, we have observed similar types of syntax errors and missing tokens in user-written queries. By replicating these patterns in a controlled and systematic way, our

dataset achieves both realism and balance across error types, which is often lacking in naturally occurring data. Moreover, we manually validate the injected errors to ensure they are semantically meaningful and representative. Thus, the semi-synthetic approach enables scalable and reproducible evaluation, while still retaining practical relevance to real-world SQL tasks.

4.3 Large Language Models

We use several state-of-the-art LLMs, briefly described below.

GPT4. OpenAI’s GPT4, launched in 2023, boasts over 200 billion parameters, with significant improvements in contextual understanding and reasoning over its predecessor. It was trained on a larger and more diverse dataset, enhancing its performance across a variety of language tasks [43].

For interacting with GPT4, we utilized both ChatGPT 4 and an API. The API was specifically used for obtaining binary answers in the Missing Token Task because it allows for precise control over parameters—ChatGPT, in contrast, often produces lengthier responses. However, while using the API is more costly due to additional charges per token, ChatGPT also incurs a monthly fee of around CAD 33. Despite this, the API remains advantageous for certain tasks due to its extensive customizable parameters, such as selecting the model (either GPT-4 or GPT-3.5), adjusting conversation histories, setting maximum tokens to one, and controlling output determinism with settings like `temperature=0` and `top_p=1`. This configuration ensures that the output is limited to a single token, as defined by `max_tokens`. The parameter `temperature=0` is used to ensure that the output from GPT4 is very predictable and consistent. This setting is particularly useful when a binary answer, such as “YES” or “NO,” is required, as it eliminates variability in the responses. Conversely, setting the temperature close to 1 would result in more diverse and varied responses from the LLM, as it allows for a broader selection of probable tokens.

GPT3.5. Released by OpenAI in late 2022, GPT3.5 consists of 175 billion parameters and is trained on a large corpus including Common Crawl, Wikipedia, and various books and academic texts. It is designed to handle diverse NLP tasks [5]. We applied the same processing approach to interact with GPT3.5, merely changing the model parameter from GPT4 to GPT3.5.

Gemini. Developed by Google and introduced in 2024, Gemini prioritizes both accuracy and safety in AI interactions, with a strong emphasis on ethical AI. It has an estimated parameter count of 50 billion and is trained on vast multimodal datasets, handling both text and visual data. This focus on multimodal capabilities and alignment with human values differentiates it from models primarily designed for text-based tasks [2]. We opted to use the chat interface for interacting with Gemini because there were country restrictions on using Gemini’s API at the time.

Llama3. Meta’s Llama3, released in 2023, is available in versions up to 70 billion parameters. Trained on trillions of tokens from general-purpose datasets, it is designed for efficiency and scalability. LLaMA’s focus on broad, general-purpose language tasks makes it ideal for deployment in resource-constrained environments, where maintaining performance is crucial [57]. We used the API for interaction with Llama3, downloading Llama3 locally and then utilizing

the API. We also employed the API for binary responses in the Missing Token Task to ensure consistency with other LLMs.

MistralAI. Launched in 2024, MistralAI is optimized for high accuracy with a smaller footprint of 16 billion parameters. It is trained on a wide range of datasets, with an emphasis on domain-specific content and multilingual capabilities. MistralAI is particularly suited for specialized tasks, that require deeper understanding in areas such as SQL and other structured data, offering a balance of computational efficiency and domain-targeted performance [38].

LLMs achieve high performance due to their extensive training on datasets ranging from hundreds of billions to trillions of tokens. The trend in LLM development is to leverage larger datasets and more complex architectures to continually improve generalization across diverse tasks [5, 43, 57, 38].

4.4 Refining LLM Interactions

Interacting with LLMs requires careful attention to both input prompts and processing of their responses. By tuning prompts, we guide the models toward generating more accurate and relevant outputs. However, the responses also require post-processing because they are often not provided in a straightforward format necessary for our tasks, such as a simple label. Post-processing involves extracting the necessary information from potentially verbose or complex responses, and ensuring that it fits the specific format required for evaluation.

Prompt Tuning. Designing and refining input prompts to guide LLMs toward accurate responses is particularly important for complex tasks, where well-crafted prompts can significantly improve model performance [61, 49, 37]. In our thesis, prompt tuning is essential to effectively handle the intricacies of SQL syntax and semantics. Our tuning process involved two key steps:

1. *Prompt Generation and Refinement.* We used LLMs to generate a variety of prompt candidates, which were then manually refined to ensure clarity and alignment with our task objectives [59, 3].
2. *Mock Experiments.* We conducted mock experiments with a subset of data to evaluate the effectiveness of each prompt. The top-performing prompts from these tests were selected for full-scale experiments.

Following this approach, we developed a set of task-specific prompts to extract meaningful responses from the models. We used a single, consistent prompt across all LLMs to ensure a fair and unbiased evaluation. The goal was to compare the models based solely on their inherent capabilities, without the influence of prompt tuning or optimization. This decision was particularly important because we observed that some models—such as Gemini—exhibited significant variability in their outputs depending on how the prompt was phrased. To avoid introducing such inconsistencies and to maintain comparability, we adopted a fixed prompt design. This approach not only supports a controlled experimental setup but also reflects practical, zero-shot scenarios where users often rely on generic instructions without extensive prompt engineering.

These prompts were tailored to each experimental task and varied in complexity, addressing challenges such as syntax error detection, query equivalence, and runtime estimation:

- `syntax_error` and `syntax_error_type`. Does the following query contain any syntax errors? If so, explain the error. [query]
- `miss_token`, `miss_token_type`, and `miss_token_loc`. Does the following query have any syntax errors? (yes/no) If yes, is there a missing word? (yes/no) If yes, what is the type of the missing word? If yes, what is the missing word? If yes, what is the position of the missing word? (Provide the word count position where the word is missing.) [query]
- `query_equiv` and `query_equiv_type`. Are the following two queries equivalent (do they produce the same results on the same database schema)? If yes, why are they equivalent? [query 1, query 2]
- `performance_pred`. Does the following query take longer than usual to run? [query]
- `query_exp`. Provide a single statement describing this query: [query].

The prompts listed above reflect the outcomes of our prompt tuning approach, which was specifically designed to address the SQL tasks in our study. For example, in `miss_token`, the prompt used was: “Does the following SQL query have any missing words (yes/no)? If a word is missing, what type of word is it? At which position is the missing word located (word count)? [query]” This prompt was designed to draw out detailed, step-by-step responses from the model, addressing both `miss_token`, `miss_token_type`, and `miss_token_loc`. In contrast, for `performance_pred`, we used a simpler, binary prompt: “Does this query take a long time to run: [query]?” This was chosen to focus the model’s output on a straightforward performance assessment. The full list of initial prompts and the selected prompts used in the mock experiments are reported in our repository.

Handling LLM Output. Despite providing clear instructions, it is necessary to post-process the LLM output results. LLMs often (is necessary, even after providing clear instructions, as some LLMs still) produce lengthy and verbose responses that require careful extraction of relevant information. For example, in the prediction task described above, while most LLMs respond with a binary “yes” or “no,” they often provide explanations about why the query takes a long or short time to execute. Similarly, in the `miss_token` task, the responses are not always formatted in a structure that aligns with our evaluation criteria. To address this, we rely on a combination of manual processing and automated scripts. Manual processing involves reading through the responses to extract the specific information required, e.g., isolating the “yes” or “no” from the rest of the explanation. To expedite this, we use scripts to detect common response patterns and automatically extract the relevant portions when the responses follow predictable structures. However, for more complex or less structured outputs, manual intervention is still necessary to ensure accuracy. This allows us to format the LLM outputs consistently, and to evaluate their performance effectively.

Zero-Shot, Few-Shot, and Fine-Tuning. There are three main approaches for using LLMs are zero-shot, few-shot, and fine-tuning. In the zero-shot learning approach, the model is used to make predictions or generate responses without any additional training specific to the task at hand. This means the model uses only its pre-existing knowledge and understanding, which it acquired during its initial training phase. For example, you can ask a language model a

question or request it to perform a task that it has not explicitly been trained on, and it will attempt to infer the best possible answer based on its general capabilities.

Few-shot learning involves providing the model with a small number of examples at inference time to help it understand the context or the specific nature of the task. This does not involve retraining the model; instead, these examples are used as a prompt that guides the model on how it should respond. The examples act like a mini-training session where the model quickly adapts its responses based on the new information presented in these examples.

Fine-tuning is a more involved process where the model is further trained on a large dataset that is specific to the task or domain of interest. This process adjusts the weights of the model across potentially all layers, refining its responses and predictions to be more accurate for specific topics or types of tasks. Fine-tuning requires computational resources and more data but results in a model that is better specialized to the requirements of the task.

In our experiments, we focused exclusively on zero-shot learning to assess a model's ability to detect syntax errors, evaluate query equivalence, and predict query runtime costs. Our goal was to study the models in their raw form, without introducing additional task-specific information, which reflect real-world scenarios where such data may not always be available.

Few-shot learning provides a model with few examples to improve performance for example, providing examples of pairs of queries that are either equivalent or not helps the model understand the nuanced differences or similarities that define equivalence, improving its judgment.

Fine-tuning further trains a model on task-specific datasets to improve accuracy, for example, by fine-tuning on datasets that include a wide variety of SQL errors, the model can learn specific error patterns and the corrections required, enabling it to perform more accurately in real-world applications where error detection is crucial. For fine-tuning, large datasets are required; however, SQL-related datasets are not easily found, so fine-tuning is not extensively practiced. But in the area of text-to-SQL, there are many labeled datasets available, which has facilitated more work in this field [19].

Although both approaches can help in situations where a model's initial performance is lacking, we did not use either method in our study. Our goal was to evaluate LLMs with minimal additional training to reflect their performance in environments where limited labeled data are available.

4.5 Baselines for Experimental Evaluation

We employ baselines to contextualize the results from LLMs and evaluate their strengths and weaknesses. For `performance_pred`, we fine-tuned the BERT model to predict SQL query execution times. BERT, a pretrained transformer widely used for NLP tasks, was adapted to our dataset of 10,000 randomly selected SQL queries from the SDSS 2024 workload, annotated with binary labels ('high' or 'low' runtime). Binary cross-entropy was employed as the loss function to fine-tune the model for this task. To train the model, the learning rate is set to 0.00003. The batch size per device for training is 8, and for evaluation, it is 16. The number of training epochs is 3, and the weight decay is set to 0.01. No data leakage occurred, as distinct queries from SDSS 2023 were used for testing. For `syntax_error` and `miss_token`, we used ANTLR, which stands for ANother Tool for Language Recognition, is a powerful

tool utilized here to construct robust parsers [44]. This is a general-purpose parser widely used for SQL, as a baseline. ANTLR detects syntax errors and missing tokens without relying on schema information, ensuring fair comparison with LLMs. Using DBMSs with workload-specific schemas for parsing was deemed impractical for SQLShare (diverse databases) and SDSS (large schema with limited access). Moreover, DBMS-based baselines would create an unfair advantage by incorporating unavailable schema details. The parser baseline, denoted as SQLParser, provides binary outputs for `syntax_error` and `miss_token` and extracts error messages to infer `miss_token_loc` details. This parser is essential for breaking down SQL queries into their constituent elements and checking each element against SQL grammar rules to determine their correctness.

For ANTLR output processing, we used two types of Java code. For binary tasks, such as syntax errors and missing token tasks, we created a txt file containing all SQL queries from specific dataset(SDSS, SQLShare or Join-Order). Each SQL query in the file starts with the number 1. This txt file is then passed to the Java program, which determines whether each query is syntactically correct or not. For identifying missing positions, ANTLR generates a syntax tree and identifies the next word where the missing word should be located. The parser does not classify specific syntax errors or missing token types as defined in this study, and we limit the baseline comparison to the binary tasks and `miss_token_loc`.

Case	Model	SDSS			SQLShare			Join-Order		
		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Syntax Error	GPT4	0.98	0.95	0.97	<u>0.94</u>	0.93	0.93	0.95	<u>0.91</u>	0.93
	GPT3.5	0.94	0.85	0.89	0.91	0.86	0.89	<u>0.93</u>	0.81	0.86
	Llama3	<u>0.95</u>	0.76	0.84	0.92	0.81	0.86	0.95	0.65	0.77
	MistralAI	0.93	<u>0.91</u>	<u>0.92</u>	0.92	<u>0.91</u>	<u>0.92</u>	0.85	0.94	<u>0.89</u>
	Gemini	0.94	0.70	0.80	0.97	0.53	0.68	0.84	0.61	0.70
	SQLParser	0.90	0.05	0.09	0.80	0.02	0.04	0.75	0.03	0.05
Syn. Error Type	GPT4	0.96	0.95	0.95	0.89	0.88	0.88	0.90	0.89	0.89
	GPT3.5	0.87	0.85	0.85	<u>0.85</u>	<u>0.82</u>	<u>0.83</u>	0.83	0.78	0.78
	Llama3	0.83	0.79	0.79	0.79	0.76	0.76	0.78	0.67	0.64
	MistralAI	<u>0.90</u>	<u>0.88</u>	<u>0.89</u>	0.81	0.80	0.79	<u>0.86</u>	<u>0.81</u>	<u>0.82</u>
	Gemini	0.81	0.74	0.73	0.73	0.60	0.58	0.68	0.53	0.52

Table 5.1: Accuracy in `syntax_error` and `syntax_error_type`

Chapter 5

Experimental Results

In this Chapter, we present our results and analysis, with each section focusing on a primary SQL task, and its related secondary tasks.

5.1 Syntax Error Tasks

In this section, we present results for the two related tasks of `syntax_error` and `syntax_error_type` using figures and table 5.1 .

syntax_error. Table 5.1 (top) shows the comparative accuracy on the `syntax_error` task. The best-performing model is highlighted in bold, and the second-best is underlined. GPT4, GPT3.5, and MistralAI perform well, while Llama3 and Gemini struggle. This may be because Llama3 is trained on general-purpose text, and Gemini focuses more on AI ethics and multimodal tasks, meaning both have less specific knowledge of SQL compared to the other models.

Across all models, recall tends to be lower than precision, suggesting that the models are more conservative in detecting errors, missing some existing syntax errors (lower recall) but

making fewer incorrect claims about errors (higher precision). One possible explanation is that these models may have been trained more extensively on correct SQL queries, with less exposure to syntactically incorrect examples. This precision-recall imbalance is particularly pronounced in Llama3 and Gemini, which exhibit significantly lower recall, resulting in reduced F1 scores as well.

SQLParser, the baseline for this task, achieves high precision but very low recall across all datasets. This is because the syntax errors considered in `syntax_error` are advanced and often cannot be detected using only the grammatical rules of SQL in such parsers. For example, `nested-mismatch`, where the return type of a subquery does not match the expected type in the outer query, is beyond the capabilities of SQLParser. Similarly, most parsers, including the one used in our experiments, cannot verify the `having` clause for errors like `aggr-having`. The high precision is mainly due to simpler error types, such as `alias-undefined`, which resemble missing tokens and can be detected by parsers. These results emphasize the limitations of traditional parsers and affirm the superior ability of LLMs to effectively detect both simple and advanced syntax errors, as well as other complex cases.

An important question is when and why LLMs fail in `syntax_error`. To explore this, we examined two hypotheses: first, that failures are related to the syntactic properties of queries, such as `word_count` or `table_count`; and second, that they are linked to specific types of syntax errors, such as `aggr-attr` or `nested-mismatch`, as discussed in Section 4.1. We applied the same analysis to other tasks while testing these hypotheses.

For the first hypothesis, we analyzed the distribution of syntactic properties across four categories: true positive (TP), true negative (TN), false positive (FP), and false negative (FN). Figure 5.1 illustrates this for `syntax_error` in SDSS, showing query distributions by `word_count`. The numbers below each category represent the average (top), median (middle), and total number of queries (bottom). Figures 5.1a and 5.1b show similar data for Llama3 and Gemini but 5.1c shows a different data for GPT3.5.

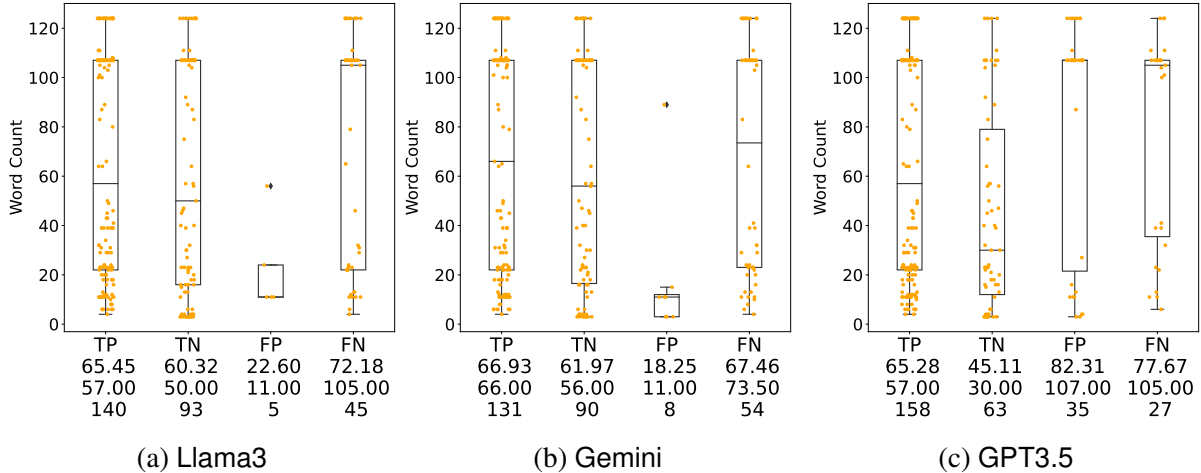


Figure 5.1: Relationship between `word_count` and model failure in `syntax_error` for SDSS. The three numbers (e.g., 65.45, 57.00, 140) represent the average and median query length and the number of queries in the category (TP). The orange scatter points represent queries plotted on the y-axis by their length, showing the length distribution per category.

To explore the correlation between query length (`word_count`) and model failure, we compared TP and FN (for queries with errors) as well as TN and FP (for queries without errors) while concluding when there are significant queries in each category. For example, in Figure 5.1a, the TP and FN categories have sufficient queries (140 and 45, respectively) to observe a pattern: the FP queries tend to be significantly shorter (average 22.60 vs 72.18, median 11 vs 105). A opposite trend is seen when comparing FP and FN, where FP queries are shorter but FN queries are very long. This trend is also observed while comparing TP and FN in Gemini in Figure 5.1b. On the other hand, in Figure 5.1c, we observe that both FP and FN occur due to the long queries which is unusual for Llama3 and Gemini. Overall, these findings suggest a correlation between query length (`word_count`) and failure in `syntax_error`, with longer queries being more prone to misclassification. We did not observe a similar pattern for any other syntactic properties across models or datasets, indicating that `word_count` is the most significant factor influencing failure likelihood in this task.

For the second hypothesis, Figure 5.2 presents the proportion of queries in FN for each type of syntax error, where a larger bar indicates that detecting errors of that type has been more challenging for the models. The results for SDSS (Figure 5.2a) suggest that type mismatch errors (`nested-mismatch` ■ and `condition-mismatch` ■) are particularly difficult for all models to detect. This is expected as the workload involves queries with many conditions for which the type of operands could be difficult to tell. For SQLShare (Figure 5.2b), ambiguous alias (`alias-ambiguous` ■) errors are more problematic, which is expected given the large number of schemas and varied table aliases used in these queries. Lastly, in Join-Order (Figure 5.2c), errors involving mismatch in using nested queries (`nested-mismatch` ■) are the most frequently missed by the models since similar to SDSS the queries in Join-Order also have lengthy conditions in the `WHERE` clauses.

syntax_error_type. Table 5.1 (bottom) presents the weighted accuracy for `syntax_error_type`, which considers the six types of syntax errors (see Section 4.1). The strong performance of GPT4 and MistralAI, and GPT3.5 in detecting syntax errors also extends to identifying error types, while Llama3 and Gemini continue to perform less effectively, as expected. Overall, results for `syntax_error_type` are lower than for `syntax_error`, reflecting the increased difficulty of this task. Another key observation is that all models show lower performance on the SQLShare dataset, likely due to its more complex schema, which makes identifying the type of syntax errors more challenging.

Takeaways: The analysis of `syntax_error` and `syntax_error_type` shows that GPT4, MistralAI, and GPT3.5 outperform Llama3 and Gemini, likely due to differences in training focus. Longer queries are more prone to errors, and the types of syntax errors the models struggle with largely depend on the specific dataset.

5.2 Missing Token Tasks

Regarding missing token, we start by `miss_token`, and then present results related to `miss_token_type` and `miss_token_loc`.

miss_token. Table 5.2 (top) presents the accuracy of various LLMs in `miss_token`. Accuracy is higher compared to `syntax_error`, as `miss_token` is a simpler task. A notable change is

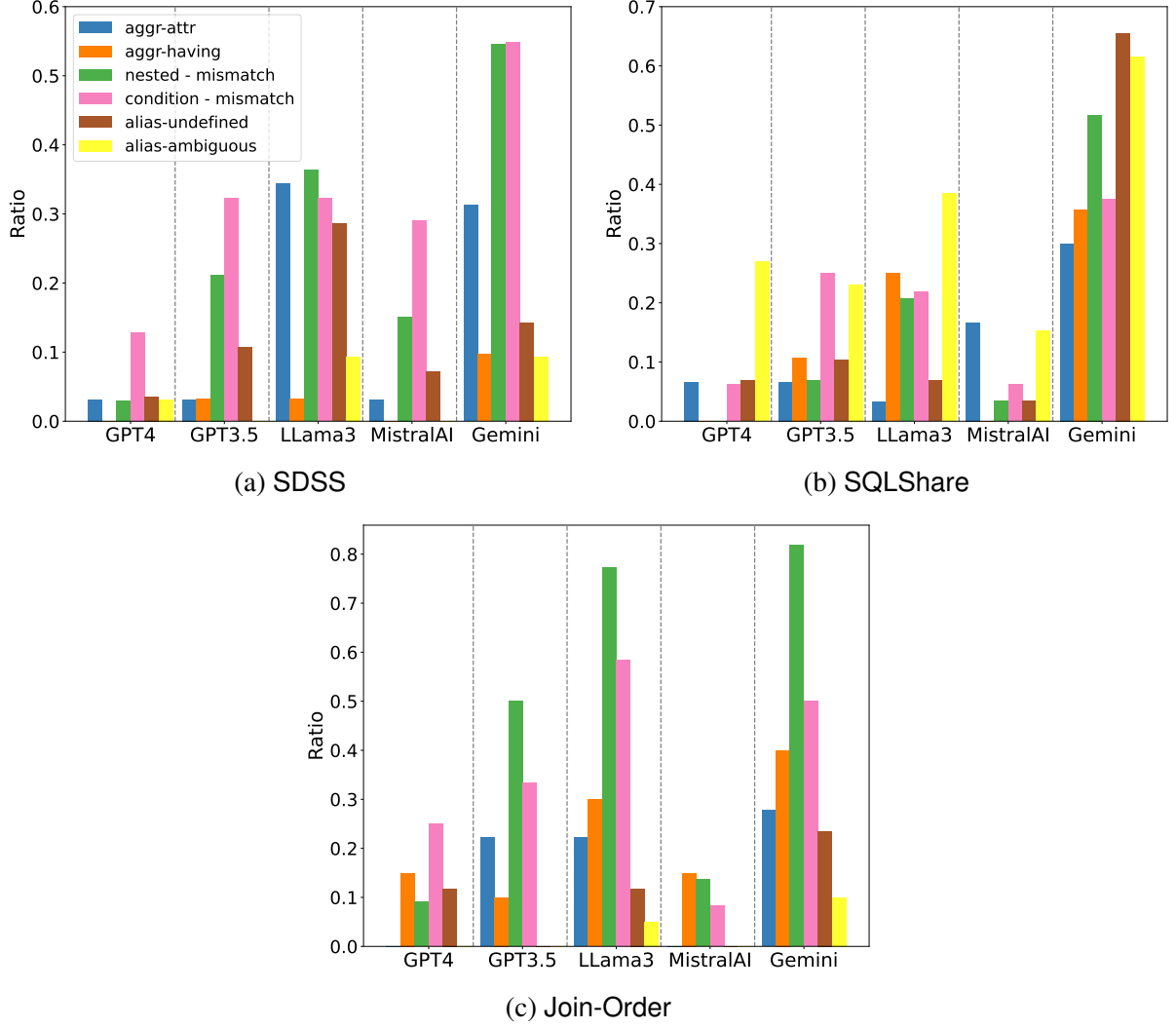


Figure 5.2: Relationship between syntax error type and FN in `syntax_error`.

Llama3's improved performance in this task. This can be attributed to the fact that detecting missing tokens relies more on general pattern recognition, which is less specialized for SQL. Llama3's broader training in recognizing patterns likely helps it improve in this context. Overall, recall remains lower than precision in `miss_token`, similar to `syntax_error`, likely because the models are more conservative in detecting errors, as explained previously.

SQLParser easily identifies queries with missing tokens. This is because the simple grammatical rules used in SQL parsers like SQLParser effectively detect missing tokens during syntax tree construction.

We investigated the relationship between LLM failures in the `miss_token` task and the syntactic properties of queries. Figure 5.3a shows that, for GPT3.5 on the SQLShare dataset, query length (`word_count`) is correlated with failures, with an average `word_count` of 57 in FN compared to 27 in TP. We also examined other properties such as predicate count (`predicate_count`), nestedness level (`nestedness`), and table count (`table_count`), as seen in Figures 5.3b, 5.3c, and 5.3d. In all cases, the average values for FN are significantly higher

Case	Model	SDSS			SQLShare			Join-Order		
		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Missing Token	GPT4	0.99	0.97	0.98	0.98	0.96	0.97	1.00	0.97	0.99
	GPT3.5	0.92	0.92	0.92	<u>0.97</u>	0.88	0.93	<u>0.98</u>	<u>0.94</u>	0.96
	Llama3	0.96	0.94	0.95	<u>0.91</u>	0.92	0.91	<u>0.97</u>	<u>0.94</u>	0.96
	MistralAI	0.99	0.86	0.92	0.96	0.87	0.91	1.00	<u>0.94</u>	<u>0.97</u>
	Gemini	0.99	0.76	0.86	0.98	0.68	0.80	0.97	0.69	0.81
	SQLParser	<u>0.97</u>	<u>0.95</u>	<u>0.96</u>	0.94	<u>0.93</u>	<u>0.94</u>	0.97	0.93	0.95
Token Type	GPT4	0.94	0.94	0.94	0.91	0.89	0.90	0.98	0.97	0.98
	GPT3.5	0.76	0.75	0.75	0.75	0.71	0.73	0.84	0.82	0.82
	Llama3	0.88	<u>0.85</u>	<u>0.86</u>	0.78	0.69	0.72	0.87	0.82	0.84
	MistralAI	<u>0.89</u>	<u>0.85</u>	<u>0.86</u>	<u>0.82</u>	<u>0.75</u>	<u>0.78</u>	<u>0.93</u>	<u>0.88</u>	<u>0.90</u>
	Gemini	0.63	0.63	0.54	0.75	0.53	0.57	0.44	0.60	0.39

Table 5.2: Accuracy for `miss_token` and `miss_token_type`

than for TP (1.80 vs 0.90 in 5.3b, 0.44 vs 0.05 in 5.3c, and 1.92 vs 1.33 in 5.3d). However, due to the small number of FP queries, no definitive conclusions can be drawn for that category.

We now shift our analysis to the impact of the missing token type on the performance of LLMs in `miss_token`. We examined the breakdown of FN by token type, as shown in Figure 5.4, similar to our analysis for `syntax_error`. A key observation in SDSS is that the most frequent type of failure occurs for keyword (■). This is likely because SDSS contains a diverse set of query types with a higher occurrence of keywords compared to SQLShare and Join-Order. In SQLShare, the most challenging missing token types are aliases and tables (■ and ■), which can be attributed to the presence of many small databases with numerous tables and various aliases in their queries. Finally, in Join-Order, there is no single token type with a notably higher failure rate, likely due to the simpler nature of the queries and the relatively low number of failures.

To better understand SQLParser’s performance, which ranked second for most workloads, we analyzed its ability to detect different types of missing tokens. We found that SQLParser struggled particularly with missing columns and aliases, likely because these rely heavily on schema information, which cannot be fully addressed using only the grammatical rules of an SQL parser.

`miss_token_type`. We reported the weighted average accuracy values in Table 5.2 (bottom), with weights based on the number of queries for each type. The results indicate that `miss_token_type` is more challenging than `miss_token` across all LLMs, as evidenced by the reduced accuracy. The lowest accuracy is observed in SQLShare, which is expected due to its complex schema compared to SDSS and Join-Order. Conversely, Join-Order shows the highest accuracy, reflecting its simpler schema. Notably, MistralAI consistently achieves the second-best performance. This is interesting as Llama3 was the second in `miss_token`, which suggests although Llama3 is better at detection due to its strength in detecting general patterns, MistralAI is better at SQL-related pattern recognition, correctly deciding the error type for more queries.

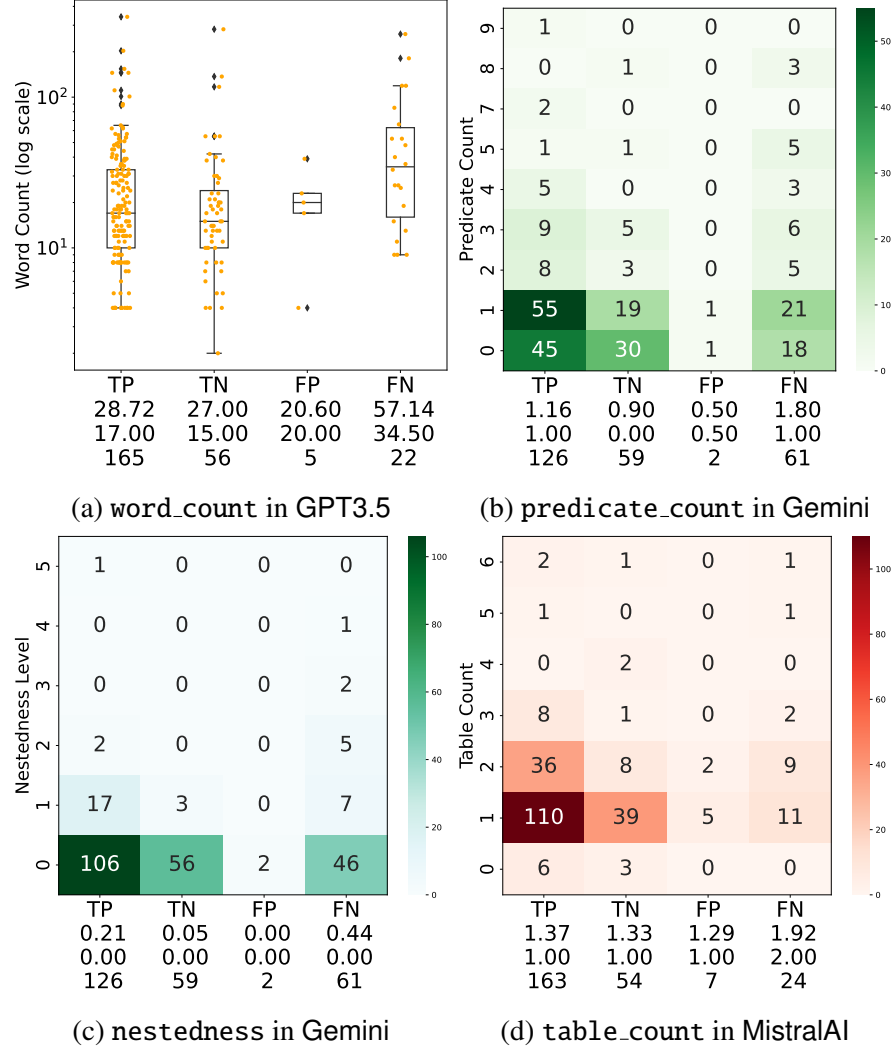


Figure 5.3: LLMs' failure in miss_token for SQLShare.

miss_token_loc. Table 5.3 compares the performance of various LLMs in predicting the location of the missing token across SDSS, SQLShare, and Join-Order. The key metrics are Mean Absolute Error (MAE) and Hit Rate (HR), where lower MAE and higher HR indicate better performance.

GPT4 consistently achieves the best results with the lowest MAE and highest HR across all datasets. Llama3 performs well in SQLShare but shows weaker results elsewhere. GPT3.5 and MistralAI provide reasonable performance but with higher MAE and lower HR, reflecting less precision and accuracy.

SQLParser performs comparatively well and is the second best in most evaluations, leveraging syntax tree construction to accurately detect missing tokens and their locations in most queries.

Most models correctly predict the exact location at least 30% of the time, except GPT3.5 in SDSS, where the HR drops to 25%. Longer queries, especially in SDSS, contribute to higher MAE values, making precise location prediction more difficult.

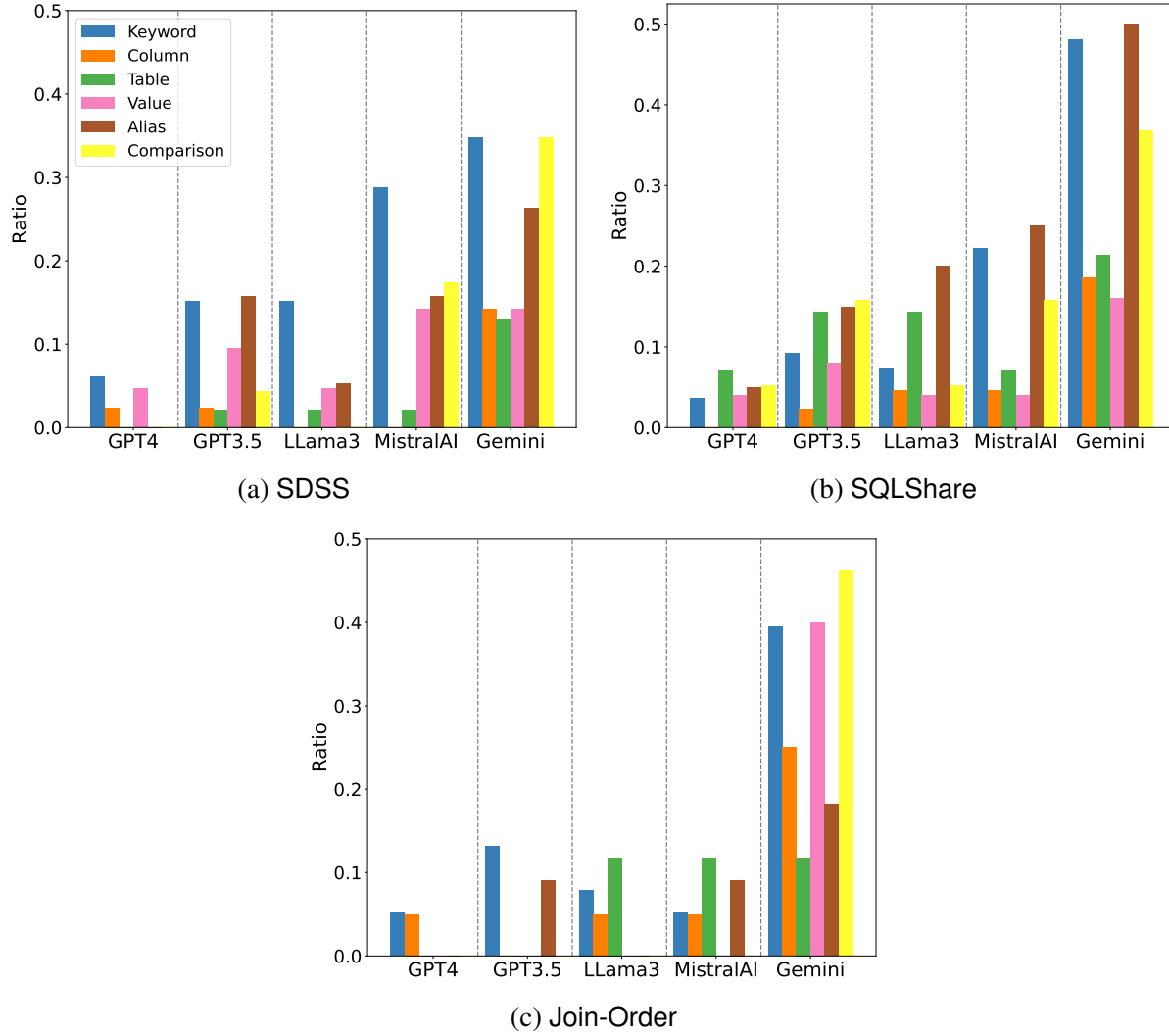


Figure 5.4: Relationship between missing token type and FN in `miss_token`.

Takeaways: All models perform better over the missing token tasks than syntax error detection, as missing token identification seems to be a simpler task related to learning frequent patterns. LLama3 shows improved performance due to its broad training in pattern detection. More complex queries tend to increase prediction errors, where complexity is related to different syntactic properties, such as `word_count`, `predicate_count`, `nestedness`, and `table_count`.

5.3 Query Performance Prediction

Table 5.4 shows the performance metrics for the SDSS dataset on the `performance_pred` task. GPT4 achieves the best results, followed by GPT3.5 and LLama3, which perform similarly. MistralAI and Gemini show lower overall performance. Across all models, recall is generally higher than precision, likely due to positive bias. LLMs tend to produce overly optimistic responses, in this case predicting that queries will take longer to run. Additionally,

Model	SDSS		SQLShare		Join-Order	
	MAE	HR	MAE	HR	MAE	HR
GPT4	4.69	0.56	3.96	0.63	3.45	0.57
GPT3.5	17.71	0.25	7.71	0.42	14.31	0.39
Llama3	15.60	0.33	<u>7.57</u>	0.40	13.11	0.39
MistralAI	18.09	<u>0.36</u>	8.58	0.42	9.92	0.40
Gemini	19.78	0.34	9.79	0.38	20.22	0.32
SQLParser	<u>10.31</u>	0.35	7.63	<u>0.45</u>	<u>8.24</u>	<u>0.42</u>

Table 5.3: MAE and Hit Rate (HR) for `miss_token_loc`

Model	Prec.	Rec.	F1
GPT4	0.88	0.93	0.90
GPT3.5	0.81	0.83	0.85
Llama3	0.76	0.90	0.82
MistralAI	0.47	0.90	0.62
Gemini	0.71	0.73	0.72
BERT	<u>0.84</u>	<u>0.91</u>	<u>0.87</u>

Table 5.4: Acc. for `performance_pred`

the queries are selected from more complex, lengthy queries in SDSS, which increases the likelihood of being labeled as costly.

The baseline, BERT, performs competitively with the LLMs, ranking as the second-best model. This is expected, as it benefits from training data and can identify basic signals for costly queries. It is important to note that the LLMs achieve comparable performance without access to such training data.

As with `miss_token`, we examined the relationship between syntactic properties and failure rates for this task. The models show strong correlations between `word_count` and failure, with longer queries leading to more FP, as shown in Figure 5.5a for MistralAI. A similar trend is seen with `column_count` in Figure 5.5b. This suggests that the models mistakenly associate longer queries or those with more columns with higher execution time. Moreover, in `nestedness` and `function_count`, no LLMs showed significant correlation. Figure 5.6a illustrates the nestedness level for GPT3.5, and Figure 5.6b shows the results for Llama3. We cannot determine any reason why there are more FP and FN for the LLMs.

To compare the baseline model, BERT, we analyzed its performance on queries with different syntactic properties. The model captures the general correlation between long queries and higher evaluation costs, likely influenced by the limited examples of long-runtime queries. However, it struggles to identify short, costly queries, and some long queries that run fast. This limitation can be attributed to class imbalance, as most queries in the training data are fast (a common characteristic in SDSS), and the subset of costly queries includes very few short queries. The best LLM, GPT4, outperforms BERT without access to such training data, demonstrating their existing knowledge of SQL evaluation and their superior ability to handle these challenging query types.

Takeaways: In the query performance prediction task, GPT4 consistently shows the highest accuracy. However, all models tend to overestimate runtimes, leading to higher recall but lower precision, especially for longer and more complex queries. This suggests that improving model training with diverse query types could reduce this bias and enhance prediction accuracy.

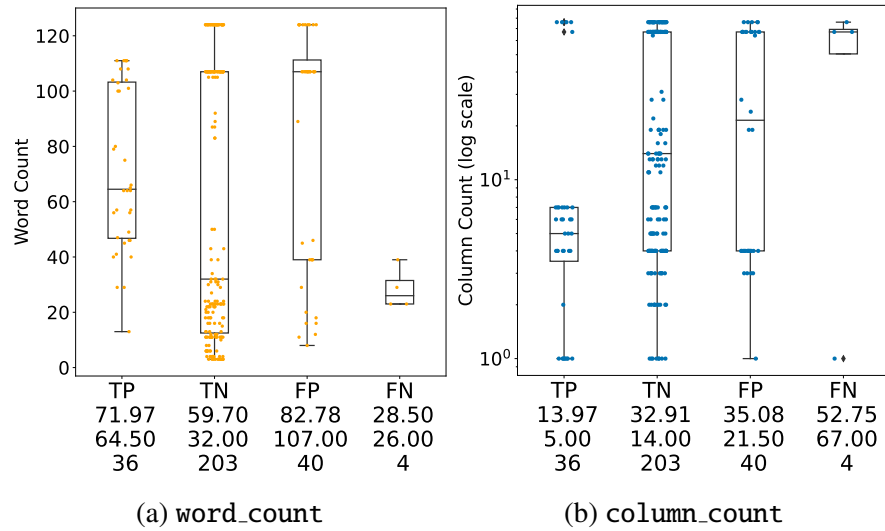


Figure 5.5: MistralAI's failure in performance_pred

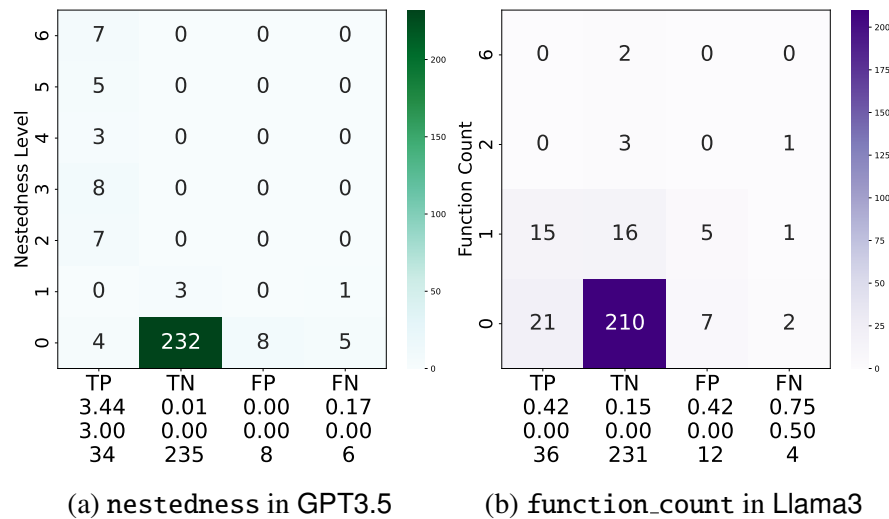


Figure 5.6: Failure in performance_pred

5.4 Query Equivalence

Table 5.5 presents the results for query_equiv (top) and query_equiv_type (bottom). For both tasks, GPT4 achieves the best performance, with GPT3.5 and Llama3 following closely but slightly less consistently. MistralAI and Gemini show more variability and generally lower

Case	Model	SDSS			SQLShare			Join-Order		
		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Equivalence	GPT4	0.98	1.00	0.99	0.97	1.00	0.99	0.91	1.00	0.95
	GPT3.5	0.87	0.99	0.93	0.96	1.00	0.98	0.83	0.99	0.90
	Llama3	0.88	1.00	0.93	0.94	0.98	0.96	0.87	0.99	0.93
	MistralAI	0.95	0.95	0.95	0.95	0.93	0.94	0.86	0.89	0.88
	Gemini	0.84	0.97	0.90	0.92	0.99	0.95	0.85	0.96	0.90
Equiv. Type	GPT4	0.99	0.99	0.99	0.98	0.98	0.98	0.95	0.85	0.83
	GPT3.5	0.97	0.91	0.91	0.96	0.92	0.94	0.90	0.78	0.77
	Llama3	0.97	0.85	0.86	0.93	0.88	0.89	0.93	0.81	0.80
	MistralAI	0.85	0.76	0.80	0.92	0.88	0.89	0.84	0.68	0.68
	Gemini	0.86	0.72	0.71	0.91	0.85	0.87	0.87	0.77	0.75

Table 5.5: Accuracy in query_equiv and query_equiv_type

scores across datasets. A positive bias (higher recall than precision) is noticeable in `query_equiv` but not in `query_equiv_type`, likely due to the latter being a multiclass task rather than binary.

Overall, `query_equiv_type` proves more challenging, with lower performance across LLMs and datasets, except for GPT4, which maintains near-perfect accuracy. This is expected, as determining equivalence is simpler than identifying the type of equivalence. Lower performance in Join-Order and SDSS compared to SQLShare suggests that longer queries make `query_equiv` more difficult.

Across all datasets, most LLMs show very few or no FN, reflected in the high recall. For example, GPT4 records FP in SDSS (5), SQLShare (4), and Join-Order (9) but has no FN. Thus, we focus on FP to identify where models fail. A common feature of FP queries is that they involve modified conditions, such as changing values in conditions. For instance, altering “WHERE run = 756 AND field = 103” to “WHERE run = 756 AND field = 200” or “WHERE run = 756 OR field = 103.” This indicates that LLMs struggle with logical reasoning and numerical manipulation, a limitation extensively discussed in the literature [16, 10, 9, 60, 11, 20] and we confirm in our study.

In addition to logical reasoning and numerical issues, these problems become more pronounced in more complex queries, such as those with longer lengths or more tables and predicates. For GPT4 in SDSS, all 5 FP involve queries over 100 words, a pattern also observed in GPT3.5 (Figure 5.7). In Join-Order, where most queries are lengthy, both FP and FN occur more frequently across all LLMs. We report only Llama3 for Join-Order, as other LLMs exhibit similar trends. Considering `table_count` as a complexity parameter, in Join-Order, all FP occur in queries with more than 8 tables. Figure 5.8 shows that in SDSS, FP occurs in queries with 5 or more predicates. Similarly, in Join-Order (Figure 5.8), all FP across models are caused by queries with over 19 predicates. We include the figure for MistralAI, as all LLMs exhibit the same pattern. These suggest that `query_equiv` and `query_equiv_type` are more difficult for complex queries (longer queries with more predicates and tables).

Takeaways: In the query equivalence task, GPT4 performs best across datasets. However, distinguishing between different types of equivalence (`query_equiv_type`) proves more difficult,

especially for Gemini and MistralAI. The errors mainly stem from challenges in understanding complex query conditions, highlighting the need for better SQL logic comprehension in LLMs.

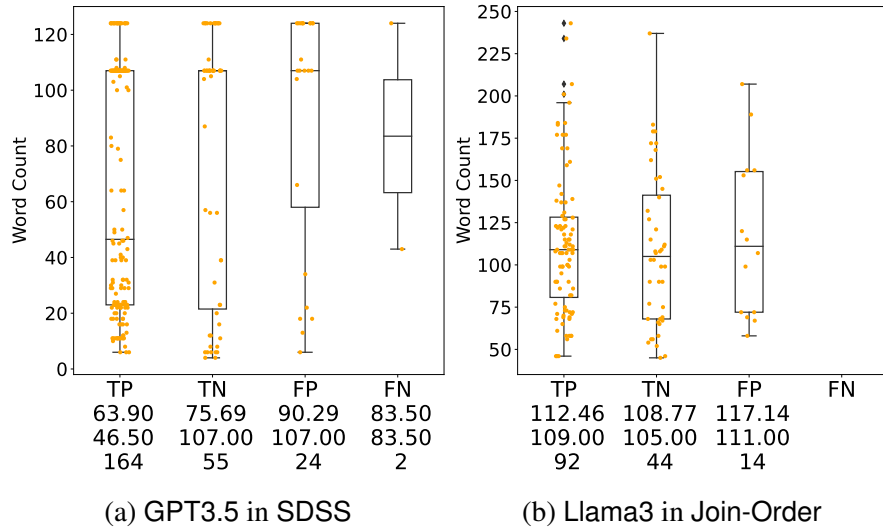


Figure 5.7: word_count and LLM failures in query_equiv.

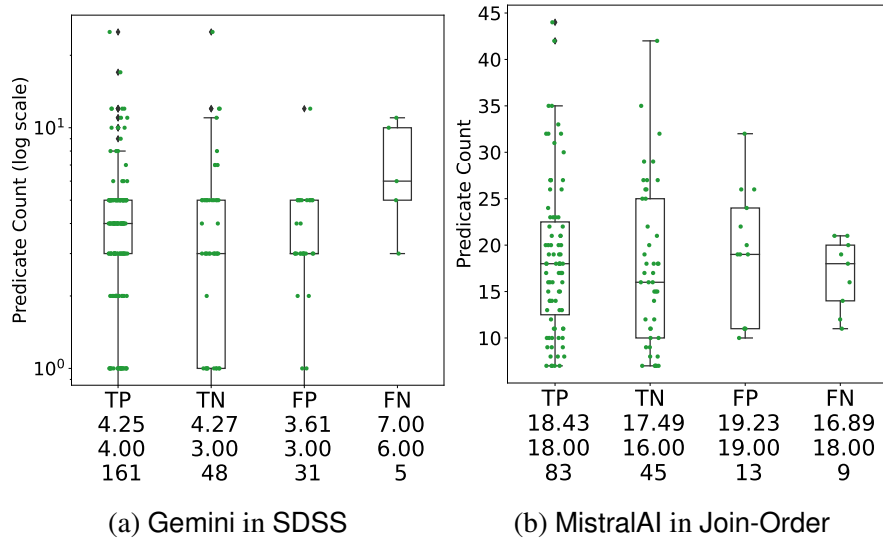


Figure 5.8: predicate_count and LLM failure in query_equiv.

5.5 Case Study: Query Explanation

We study the query_exp task and analyze several cases where LLMs failed to provide accurate explanations for SQL queries. The queries are presented in Listing 5.1. Here we present the correct ground truth descriptions from Spider, the erroneous explanations generated by the models, and briefly provide our analysis of the case:

Q15. The query finds the number of students who participate in the tryout for each college, ordered by descending count. Gemini incorrectly describes the query as: “Counts the occurrences of each unique value in the cName column.” This explanation reduces the query to a simple counting of values in the cName column, ignoring the fact that the query specifically searches for students in tryouts, which is essential to convey its full meaning.

Q16. The query identifies the maximum number of times a course enrollment result can appear in different transcripts and displays the course enrollment ID. Gemini’s explanation is: “Finds the student course ID with the highest number of occurrences.” While this partially captures the query’s purpose, it misses the query context of searching in transcripts.

Q17. The query finds the name and location of stadiums where concerts took place in both 2014 and 2015. GPT4 explains it as: “The query identifies stadiums that hosted concerts in both 2014 and 2015.” This only partially explains the query and does not include the selected attributes. This issue occurs because LLMs often focus on capturing the overall semantics of a query but overlook specific details, such as selected attributes, especially in more complex tasks.

Q18. The query retrieves the number of cylinders for the Volvo car with the least acceleration. Llama3 incorrectly explains: “This SQL query retrieves the number of cylinders of the Volvo car with the fastest acceleration.” The models misinterpret the “ORDER BY ... ASC LIMIT 1” clause, misunderstanding that the query is looking for the slowest car (lowest acceleration) rather than the fastest. Only MistralAI correctly explains this query.

Takeaways: These examples highlight a common issue with LLMs when explaining SQL queries: they often miss or misinterpret key details, particularly in tasks requiring context retention. While models may capture parts of a query, they frequently fail to provide complete and accurate explanations. This reflects known limitations of LLMs in retaining context and applying knowledge to specific scenarios [52, 33, 46].

```
-- Q15:
SELECT count(*),cName FROM tryout
GROUP BY cName ORDER BY count(*) DESC
-- Q16:
SELECT count(*),student_course_id FROM Transcript_Cnt
GROUP BY student_course_id ORDER BY count(*) DESC LIMIT 1
-- Q17:
SELECT S.name,S.loc
FROM concert AS C JOIN stadium AS S
ON C.stadium_id = S.stadium_id WHERE C.Year = 2014
INTERSECT
SELECT S.name,S.loc FROM concert AS C JOIN stadium AS S
ON C.stadium_id = S.stadium_id WHERE C.Year = 2015
-- Q18:
SELECT C.cylinders FROM CARS_DATA AS C
JOIN CAR_NAMES AS T ON C.Id = T.MakeId
WHERE T.Model = 'volvo'
ORDER BY C.accelerate ASC LIMIT 1;
```

Listing 5.1: Query statements with inaccurate explanations

5.6 Reflections on SQL Understanding

We designed our SQL tasks to probe basic skills in understanding (as described in Section 4): recognition, semantics, context, and coherence.

Demonstrated Skills. Our results show that LLMs, particularly GPT-4, perform well in tasks requiring recognition and context. For instance, in syntax error detection, the models showed high precision in recognizing violations of SQL syntax, indicating a strong ability to identify and interpret the structure of SQL queries. Similarly, missing token identification rely on the model’s context-awareness, as the ability to predict missing elements in a query depend on understanding the surrounding tokens and their relationships. This success highlights that LLMs effectively interpret the context within a query to identify missing or incorrect components.

Limitations and Shortcomings. The models were less successful at tasks requiring deeper coherence and semantic understanding. For example, query equivalence tasks which require recognizing when two syntactically different queries produce the same result, proved to be challenging, especially for longer and more complex queries. This difficulty suggests that while LLMs understand surface-level structures, they struggle with deeper semantic coherence, and logical connections within queries. For query performance estimation, the models often overestimated runtimes, indicating that they lack a nuanced understanding of how query complexity, and database-specific factors interact to affect performance.

These observations suggest that while LLMs are adept at recognizing patterns and context, their ability to fully “understand” the deeper semantics and logical coherence of SQL queries is still developing. Future work should focus on refining these models to address these shortcomings, improving their overall SQL proficiency.

5.7 Reflections and Broader Insights

Our SQL tasks were designed to evaluate the core skills of understanding—*recognition*, *semantics*, *context*, and *coherence*. This section summarizes key observations, links findings to these skills, and offers actionable insights for future research.

Key Observations. LLM performance is influenced by query complexity, task-specific challenges, and workload characteristics. Complex queries with higher word count (`word_count`), predicate count (`predicate_count`), and table count (`table_count`) lead to increased FP and FN, particularly in syntax error detection and query equivalence tasks. Longer queries exacerbate runtime overestimations in performance prediction, reflecting simplistic assumptions about execution costs. Task-specific challenges, such as ambiguous aliases in SQLShare and nested mismatches in SDSS and Join-Order, reveal some limitations in simple tasks such as `syntax_error`, where the type of error is challenging to find. Workload-specific issues amplify these challenges, with SDSS requiring extensive schema understanding, SQLShare testing semantic and contextual skills across independent schemas, and Join-Order exposing coherence issues despite simpler queries.

Core Strengths and Challenges. LLMs, particularly GPT4, excel in *recognition* and *context*, as demonstrated in syntax error detection and missing token identification. High precision in these tasks highlights strong pattern recognition and contextual awareness, making LLMs suit-

able for query validation and auto-completion. However, tasks requiring *semantics* and *coherence*, such as query equivalence and performance prediction, reveal significant gaps. Models struggle to maintain logical consistency and reason about deeper semantic relationships, underscoring the need for enhanced reasoning capabilities to handle complex queries effectively.

5.7.1 Actionable Insights and Future Directions

Building on our experimental findings, we recommend the following approaches to advance LLMs for SQL tasks:

- **Hybrid Methods:** Addressing LLM weaknesses in logical reasoning (e.g., query condition equivalence) and numerical reasoning are key to improving SQL tasks such as query equivalence and performance prediction. Combining LLMs with rule-based systems, symbolic solvers, or hybrid neuro-symbolic methods [14, 12] enhances semantic understanding, logical coherence, and reasoning capabilities, making workflows more robust and accurate.
- **Schema- and Instance-Aware Pretraining:** Incorporating schema metadata, database structures, and sample data instances during pretraining can enhance contextual and semantic reasoning. This approach is particularly beneficial for tasks such as `syntax_error`, especially in identifying complex error types that rely on schema awareness.
- **Targeted and Task-Specific Fine-Tuning:** Fine-tuning LLMs for specific tasks and error types, such as nested mismatches or ambiguous aliases, can improve robustness. For critical applications such as SQL injection detection in security, LLMs can augment traditional methods by enhancing accuracy and efficiency through their pattern recognition capabilities.

These insights emphasize leveraging LLMs' strengths in recognition and context while addressing gaps in semantics and coherence through targeted training and hybrid approaches. Advancing these areas will enhance their applicability in SQL workflows and bridge the gap between current AI capabilities and practical needs.

Chapter 6

Conclusion

This research systematically evaluated the capabilities of state-of-the-art LLMs in interpreting and "understanding" SQL, a critical component in the automation of database management and query handling. The focus was primarily on several key tasks that are pivotal to the effective deployment of AI in database systems: syntax error identification, missing token identification, query equivalence, query performance estimation, and query explanation.

Through rigorous testing, it became evident that all participating models demonstrated significant strengths in recognizing patterns and understanding context within SQL queries. GPT-4 emerged as the most capable model, outperforming its predecessors and contemporaries across a broad spectrum of tasks, but with particular prowess in managing complex SQL queries. This model's ability to navigate the intricacies of advanced SQL syntax and semantics was notable, suggesting its potential utility in sophisticated database management applications.

Conversely, other models such as GPT-3.5, MistralAI, and Llama3 also showed commendable performance, especially in pattern recognition tasks. These models efficiently identified recurring structures and themes across SQL queries, which is crucial for tasks such as query optimization and automated error detection. However, Gemini exhibited considerable difficulties, lagging behind in every category, with pronounced weaknesses in error detection—a critical aspect of SQL management.

Despite these advances, the study identified persistent challenges that curtail the utility of LLMs in real-world SQL applications. All models tested struggled with processing long and complex queries, particularly those extending beyond typical commercial use cases. A recurrent issue was the models' inability to accurately pinpoint missing tokens within queries, which is vital for error correction and learning from historical query data. Additionally, tasks that required a deep understanding of semantic coherence and logical relationships within SQL queries often saw subpar performance, highlighting a significant gap between current LLM capabilities and the nuanced demands of SQL language processing.

These insights underscore the complexity of SQL as a language that not only demands syntactic precision but also a deep understanding of the underlying logical constructs that govern databases. As AI continues to evolve, the ability of LLMs to mimic this level of understanding remains crucial for their successful integration into database management systems. The results of this study pave the way for targeted improvements in AI technologies, aiming to bridge the gap between artificial intelligence and the multifaceted realm of SQL database operations.

In the coming years, with the advancement of LLMs, several transformative changes are

expected that will enhance how users interact with databases. One significant improvement will be the Increased Accuracy in Query Translation. As models evolve, the ability to convert natural language into SQL queries or vice versa will see marked improvements in accuracy and efficiency. This enhancement means that users will experience fewer errors and enjoy a deeper understanding of their intents. Consequently, this will lead to more effective data retrieval and manipulation that truly aligns with user requests.

Additionally, More Complex Query Handling will see significant advancements as LLMs continue to evolve. The models will improve in managing more complex and nested SQL queries, which will include better handling of multiple joins, subqueries, and advanced aggregations. These aspects of SQL querying are currently challenging for many models and their enhancement will enable more sophisticated data analysis and manipulation.

Another notable development will be the Seamless Integration with Different Database Technologies. Future LLMs are expected to bridge the gap between SQL and NoSQL databases, facilitating a fluid interaction across diverse database systems. This capability will empower users to navigate various data stores effortlessly, without the need to master the specific query languages associated with each database type.

Furthermore, there will be significant advancements in Enhanced Educational Tools for SQL Learning. LLMs will play a crucial role in developing sophisticated educational resources that make learning SQL more interactive and adaptive. These tools will tailor learning experiences based on the user's progress and particular areas of difficulty, making SQL more accessible to a broader audience and enhancing user competency in database management.

These developments will not only streamline database management tasks but also expand the accessibility and utility of LLMs in professional and educational settings, fostering a more data-literate society.

6.1 Future Work

Moving forward, our research will focus on several strategies to enhance the proficiency of LLMs in handling SQL tasks, primarily through fine-tuning and dynamic prompt tuning. Fine-tuning the models with SQL-specific datasets can potentially improve their ability to parse and analyze complex queries more effectively. Additionally, dynamic prompt tuning, which adapts the model's responses based on the task's specific requirements, promises to increase accuracy and relevance of the outputs in real-time SQL query processing scenarios.

Another crucial area of future exploration is the development of methodologies to overcome the barriers that currently limit the use of LLMs in more advanced SQL tasks, such as query recommendation and optimization. By addressing these challenges, we aim to significantly enhance the models' task-specific performance, making them more practical and effective for real-world applications in database systems.

An important direction for future work is to incorporate human evaluations to better understand the quality and usefulness of LLM-generated SQL explanations. While automated metrics provide a baseline, they often fail to capture nuances that are evident to human users. Conducting a user study or survey involving both SQL experts and non-experts can help assess the clarity, correctness, and educational value of model-generated explanations. This comparison with human interpretations would offer deeper insights into how well LLMs support

real-world use cases, particularly in educational or collaborative database environments.

Another promising direction is leveraging session-level information from SQL workloads to extend LLM applications beyond single-query understanding. Many real-world database systems log sequences of related queries issued within a session, which reflect user intent, behavior patterns, and evolving query contexts. LLMs can be explored for session-aware tasks such as query clustering, summarization, or compression—identifying patterns across a session to reduce redundancy and improve efficiency. Additionally, modeling query sessions can support intelligent monitoring, such as detecting anomalies or potential cyber attacks based on unusual query sequences. Integrating session-level context could help LLMs better understand long-term dependencies, making them more effective for use in database management, security, and optimization.

Furthermore, Benchmarking and Standards Development will play a vital role in advancing LLM applications in SQL tasks. Creating benchmarks and standards would involve developing specific datasets and metrics to evaluate their performance. These could measure accuracy in query generation, efficiency in query optimization, and the model's ability to handle complex SQL constructs. Such benchmarks would facilitate consistent assessment of LLM capabilities and aid in comparing different models or approaches.

Expanding into Integration with NoSQL Databases also presents a significant opportunity. Extending the capabilities of LLMs from traditional SQL databases to NoSQL databases like MongoDB, Cassandra, or Neo4j could broaden their applicability. This adaptation requires the model to understand different data models, such as document, key-value, graph, or wide-column stores, and to generate appropriate queries or commands in those contexts.

Another crucial area of future exploration is the Integration with Database Management Systems (DBMSs). This involves developing interfaces or plugins that allow LLMs to interact directly with database software. For example, an LLM could be integrated into tools like MySQL Workbench or Microsoft SQL Server Management Studio to suggest query improvements or rewrite inefficient queries. This direct integration helps bridge the gap between natural language processing and database management, providing real-time assistance to database administrators and developers.

Lastly, LLM-Assisted Database Design could redefine how databases are structured and maintained. Future research could explore how LLMs might assist in the database design process, suggesting optimal schema designs based on data usage patterns and query performance, or automatically generating database schemas from natural language descriptions of data requirements. This could significantly streamline the database design process and ensure that database structures are optimally configured for the specific needs of their users.

By advancing the integration of LLMs with database technologies, we anticipate creating more intelligent, adaptive, and efficient systems that align better with the evolving needs of users and complex data environments.

Bibliography

- [1] Syeda Nahida Akter, Zichun Yu, Aashiq Muhamed, Tianyue Ou, Alex Bäuerle, Ángel Alexander Cabrera, Krish Dholakia, Chenyan Xiong, and Graham Neubig. An in-depth look at gemini’s language abilities. *arXiv preprint arXiv:2312.11444*, 2023.
- [2] Anthropic. Gemini: A safe and ethical large language model. *Anthropic Research*, 2024.
- [3] Ian Arawjo, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena L Glassman. Chainforge: A visual toolkit for prompt engineering and llm hypothesis testing. In *CHI*, pages 1–18, 2024.
- [4] Simran Arora, Avanika Narayan, Mayee F Chen, Laurel Orr, Neel Guha, Kush Bhatia, Ines Chami, Frederic Sala, and Christopher Ré. Ask me anything: A simple strategy for prompting language models. *arXiv preprint arXiv:2210.02441*, 2022.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [6] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45, 2024.
- [7] Gloria Chatzopoulou, Magdalini Eirinaki, and Neoklis Polyzotis. Query recommendations for interactive database exploration. In *Scientific and Statistical Database Management: 21st International Conference, SSDBM 2009 New Orleans, LA, USA, June 2-4, 2009 Proceedings 21*, pages 3–18. Springer, 2009.
- [8] Surajit Chaudhuri. An overview of query optimization in relational systems. In *PODS*, pages 34–43, 1998.
- [9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher

- Hesse, and John Schulman. Training verifiers to solve math word problems. *ArXiv*, abs/2110.14168, 2021.
- [11] Antonia Creswell, Murray Shanahan, and Irina Higgins. Selection-inference: Exploiting large language models for interpretable logical reasoning. *arXiv preprint arXiv:2205.09712*, 2022.
- [12] Artur d’Avila Garcez and Luis C Lamb. Neurosymbolic ai: The 3rd wave. *arXiv e-prints*, pages arXiv–2012, 2020.
- [13] Magdalini Eirinaki, Suju Abraham, Neoklis Polyzotis, and Naushin Shaikh. Querie: Collaborative database exploration. *IEEE Transactions on knowledge and data engineering*, 26(7):1778–1790, 2013.
- [14] Richard Evans and Edward Grefenstette. Learning explanatory rules from noisy data. *Journal of Artificial Intelligence Research*, 61:1–64, 2018.
- [15] Ju Fan, Guoliang Li, and Lizhu Zhou. Interactive sql query suggestion: Making databases user-friendly. In *2011 IEEE 27th International Conference on Data Engineering*, pages 351–362. IEEE, 2011.
- [16] Simon Frieder, Luca Pinchetti, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, and Julius Berner. Mathematical capabilities of chatgpt. *NeurIPS*, 36, 2024.
- [17] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.
- [18] Alon Y Halevy et al. Sqlshare: A platform for structured data sharing. *CIDR*, 2014.
- [19] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of llm-based text-to-sql. *arXiv preprint arXiv:2406.08426*, 2024.
- [20] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.
- [21] Magesh Jayapandian and HV Jagadish. Automating the design and construction of query forms. *IEEE transactions on knowledge and data engineering*, 21(10):1389–1402, 2008.
- [22] Minsuk Kahng, Ian Tenney, Mahima Pushkarna, Michael Xieyang Liu, James Wexler, Emily Reif, Krystal Kallarackal, Minsuk Chang, Michael Terry, and Lucas Dixon. Llm comparator: Visual analytics for side-by-side evaluation of large language models. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–7, 2024.

- [23] Nodira Khoussainova, YongChul Kwon, Magdalena Balazinska, and Dan Suciu. Snip-suggest: Context-aware autocompletion for sql. *Proceedings of the VLDB Endowment*, 4(1):22–33, 2010.
- [24] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, et al. Dynabench: Rethinking benchmarking in nlp. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4110–4124, 2021.
- [25] Won Kim. On optimizing an sql-like nested query. *ACM TODS*, 7(3):443–469, 1982.
- [26] Eugenie Yujing Lai, Zainab Zolaktaf, Mostafa Milani, Omar AlOmeir, Jianhao Cao, and Rachel Pottinger. Workload-aware query recommendation using deep learning. In *EDBT*, pages 53–65, 2023.
- [27] Bruce W Lee and JaeHyuk Lim. Tasks that language models don’t learn. *arXiv preprint arXiv:2402.11349*, 2024.
- [28] Gyeong-Geon Lee, Ehsan Latif, Lehong Shi, and Xiaoming Zhai. Gemini pro defeated by gpt-4v: Evidence from education. *arXiv preprint arXiv:2401.08660*, 2023.
- [29] Viktor Leis et al. Join order benchmark. *VLDB*, 2015.
- [30] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. Table-gpt: Table fine-tuned gpt for diverse table tasks. *ACM SIGMOD*, 2(3):1–28, 2024.
- [31] Xue Li and Till Döhmen. Towards efficient data wrangling with llms using code generation. In *DM4ML*, pages 62–66, 2024.
- [32] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.
- [33] Xing Liu et al. Attention mechanisms in large language models: A critical review. *JAIR*, 69:1021–1050, 2023.
- [34] Chaochao Lu, Chen Qian, Guodong Zheng, Hongxing Fan, Hongzhi Gao, Jie Zhang, Jing Shao, Jingyi Deng, Jinlan Fu, Kexin Huang, et al. From gpt-4 to gemini and beyond: Assessing the landscape of mllms on generalizability, trustworthiness and causality through four modalities. *arXiv preprint arXiv:2401.15071*, 2024.
- [35] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. *arXiv preprint arXiv:2104.08786*, 2021.
- [36] Yujie Lu, Xianjun Yang, Xiujun Li, Xin Eric Wang, and William Yang Wang. Llmscore: Unveiling the power of large language models in text-to-image synthesis evaluation. *Advances in Neural Information Processing Systems*, 36, 2024.

- [37] Ggaliwango Marvin, Hellen Nakayiza, Daudi Jjingo, and Joyce Nakatumba-Nabende. Prompt engineering in large language models. In *Springer DICI*, pages 387–402, 2023.
- [38] MistralAI. Mistral: New model architecture and training. *Company Blog*, 2023. Accessed: 2024-08-14.
- [39] E Nalisnick, A Matsukawa, Y Teh, D Gorur, and B Lakshminarayanan. Do deep generative models know what they don’t know? In *International Conference on Learning Representations*, 2019.
- [40] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. Using an llm to help with code understanding. In *ICSE*, pages 1–13, 2024.
- [41] Avanika Narayan, Ines Chami, Laurel Orr, Simran Arora, and Christopher Ré. Can foundation models wrangle your data? *arXiv preprint arXiv:2205.09911*, 2022.
- [42] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. Adversarial nli: A new benchmark for natural language understanding. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4885–4901, 2020.
- [43] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [44] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2013.
- [45] Karl Pearson. Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58:240–242, 1895.
- [46] Fabio Petroni et al. Contextualizing knowledge for llms: Challenges and opportunities. In *EMNLP*, pages 123–136, 2020.
- [47] Jing Qian, Hong Wang, Zekun Li, Shiyang Li, and Xifeng Yan. Limitations of language models in arithmetic and symbolic induction. *arXiv preprint arXiv:2208.05051*, 2022.
- [48] Ariel Rosenfeld and Teddy Lazebnik. Whose llm is it anyway? linguistic comparison and llm attribution for gpt-3.5, gpt-4 and bard. *arXiv preprint arXiv:2402.14533*, 2024.
- [49] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.
- [50] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. Synthetic prompting: Generating chain-of-thought demonstrations for large language models. In *International Conference on Machine Learning*, pages 30706–30775. PMLR, 2023.
- [51] Yanyan Shen, Kaushik Chakrabarti, Surajit Chaudhuri, Bolin Ding, and Lev Novik. Discovering queries based on example tuples. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 493–504, 2014.

- [52] Noah Shinn, Shuyuan Kuo, et al. Context forgetting in large language models. *arXiv preprint arXiv:2303.12345*, 2023.
- [53] Kostas Stefanidis, Marina Drosou, and Evaggelia Pitoura. You may also like” results in relational databases. In *Proceedings International Workshop on Personalized Access, Profile Management and Context Awareness: Databases, Lyon, France*, 2009.
- [54] Aaron Steiner, Ralph Peeters, and Christian Bizer. Fine-tuning large language models for entity matching. *arXiv preprint arXiv:2409.08185*, 2024.
- [55] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. Table meets llm: Can large language models understand structured table data? a benchmark and empirical study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 645–654, 2024.
- [56] Shirin Tahmasebi, Amir H Payberah, Ahmet Soylu, Dumitru Roman, and Mihhail Matskin. Transqlation: Transformer-based sql recommendation. In *2023 IEEE International Conference on Big Data (BigData)*, pages 4703–4711. IEEE, 2023.
- [57] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [58] Thinh Hung Truong, Timothy Baldwin, Karin Verspoor, and Trevor Cohn. Language models are not naysayers: an analysis of language models on negation benchmarks. In **SEM*, pages 101–114, 2023.
- [59] Li Wang, Xi Chen, XiangWen Deng, Hao Wen, MingKe You, WeiZhi Liu, Qi Li, and Jian Li. Prompt engineering in consistency and reliability with the evidence-based guideline for llms. *npj Digital Medicine*, 7(1):41, 2024.
- [60] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [61] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*, 2023.
- [62] Donald G York et al. The sloan digital sky survey. *AJ*, 2000.
- [63] Tao Yu et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *EMNLP*, 2018.
- [64] Zeyu Zhang, Paul Groth, Iacer Calixto, and Sebastian Schelter. Anymatch—efficient zero-shot entity matching with a small language model. *arXiv preprint arXiv:2409.04073*, 2024.

- [65] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International conference on machine learning*, pages 12697–12706. PMLR, 2021.
- [66] Victor Zhong et al. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.
- [67] Zainab Zolaktaf, Mostafa Milani, and Rachel Pottinger. Facilitating sql query composition and analysis. In *SIGMOD*, pages 209–224, 2020.

Curriculum Vitae

Name: Ananya Rahaman

**Post-Secondary
Education and
Degrees:** BRAC University
Dhaka, Bangladesh
2016 - 2020 B.Sc.

University of Western Ontario
London, ON
2023 - 2025 M.Sc.

Experience: Teaching Assistant, University of Western Ontario
London, Ontario, Canada
2023 - 2025

Lecturer, Primeasia University
Dhaka, Bangladesh
2021 - 2023

Publications: Evaluating SQL Understanding in Large Language Models
Ananya Rahaman, Anny Zheng, Mostafa Milani,
Fei Chiang, Rachel Pottinger
To appear in the 28th International Conference on
Extending Database Technology (EDBT),
25–28 March, Barcelona, 2025